

BPS5231

Artificial Intelligence

For

Sustainable

Building

Design

L9

Ang Yu Qian

Assistant Professor

L09.1

Supervised 8

Autoencoders

Variational Autoencoders

L09.2

Supervised 9

Generative Adversarial Networks

L09.1

Supervised 8

Autoencoders

Variational Autoencoders

L09.2

Supervised 9

Generative Adversarial Networks

Today, we will cover

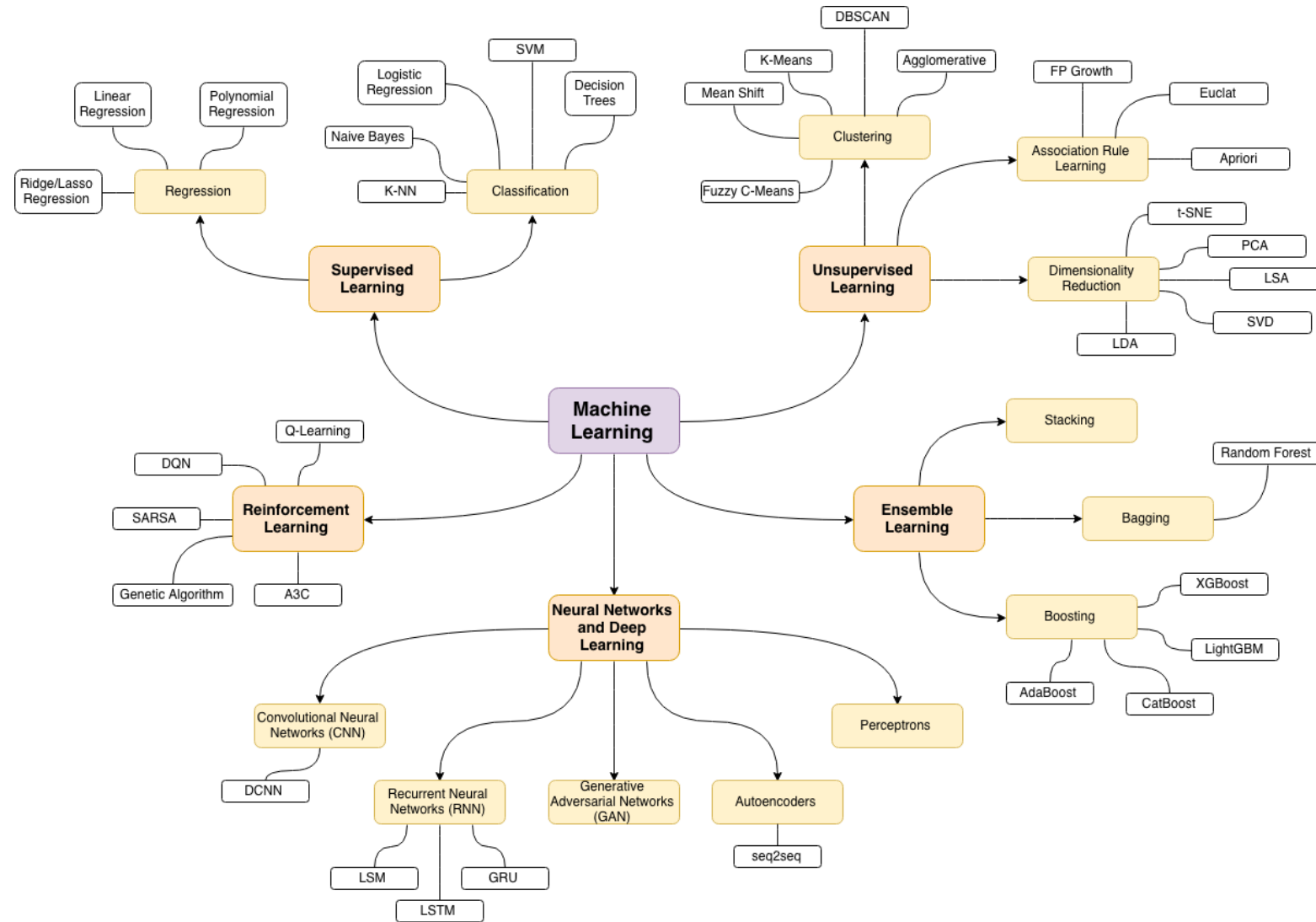
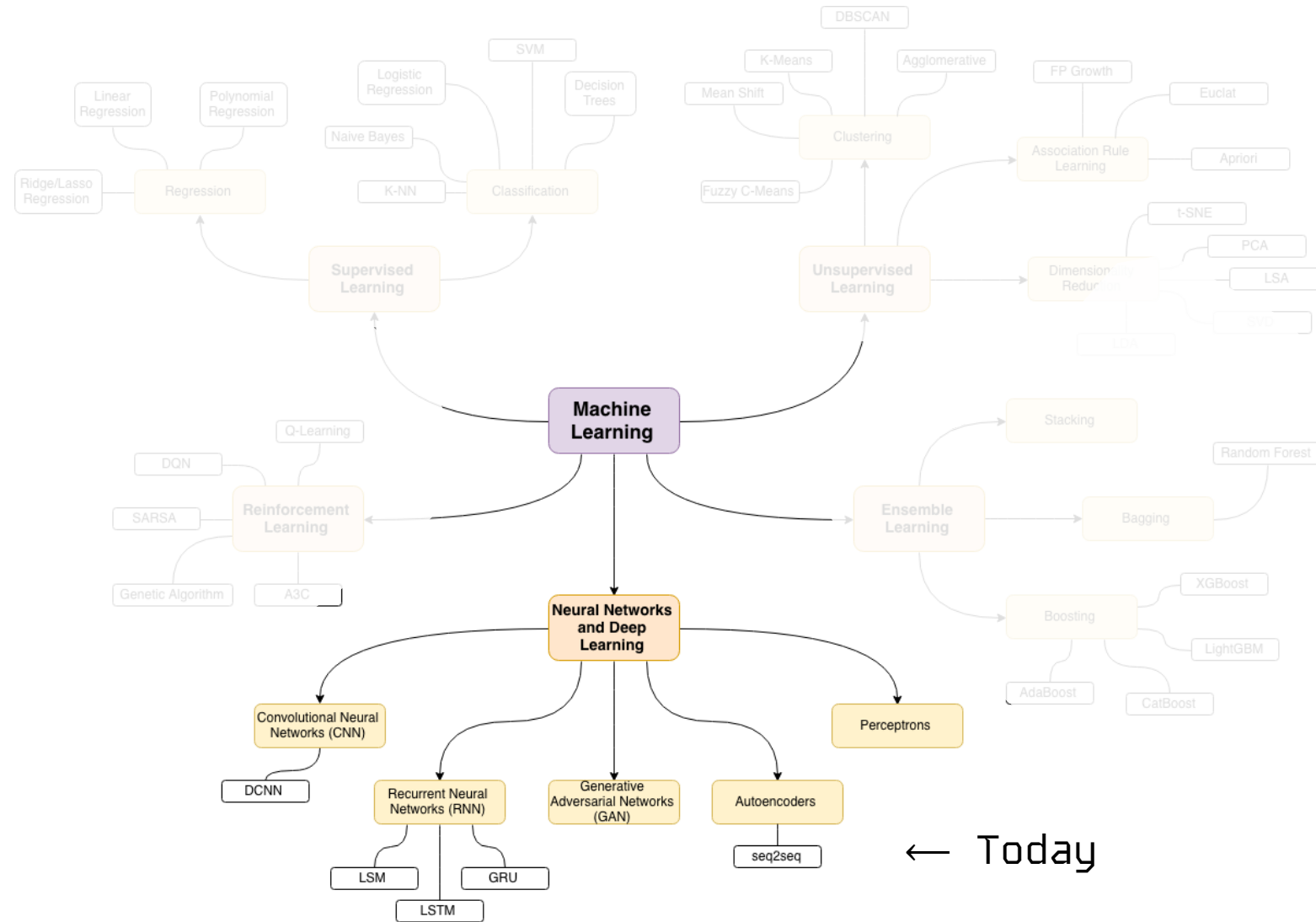


Image from Anil Jain, Google

SUPERVISED 8

SUPERVISED 9

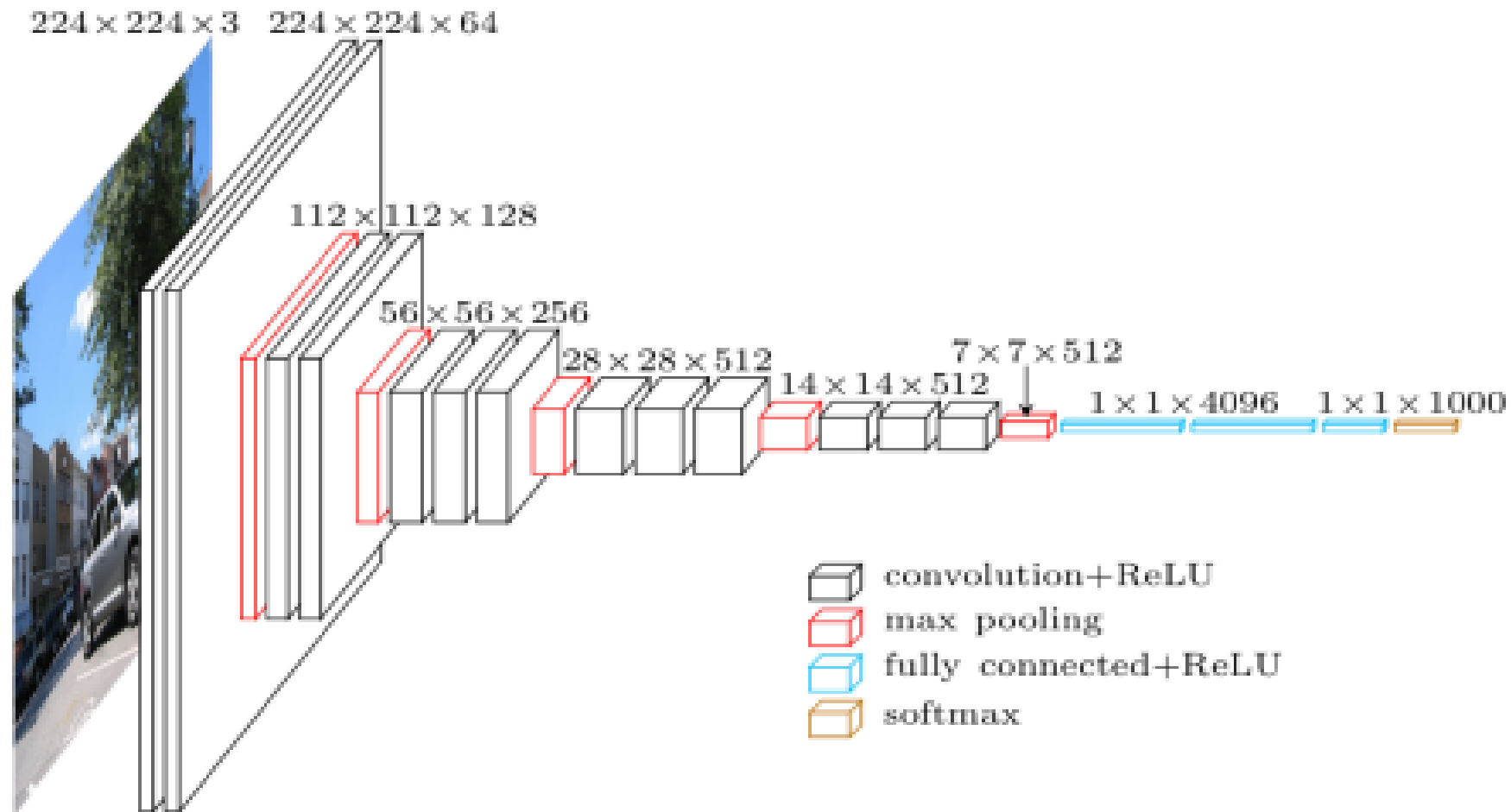
Today, we will cover



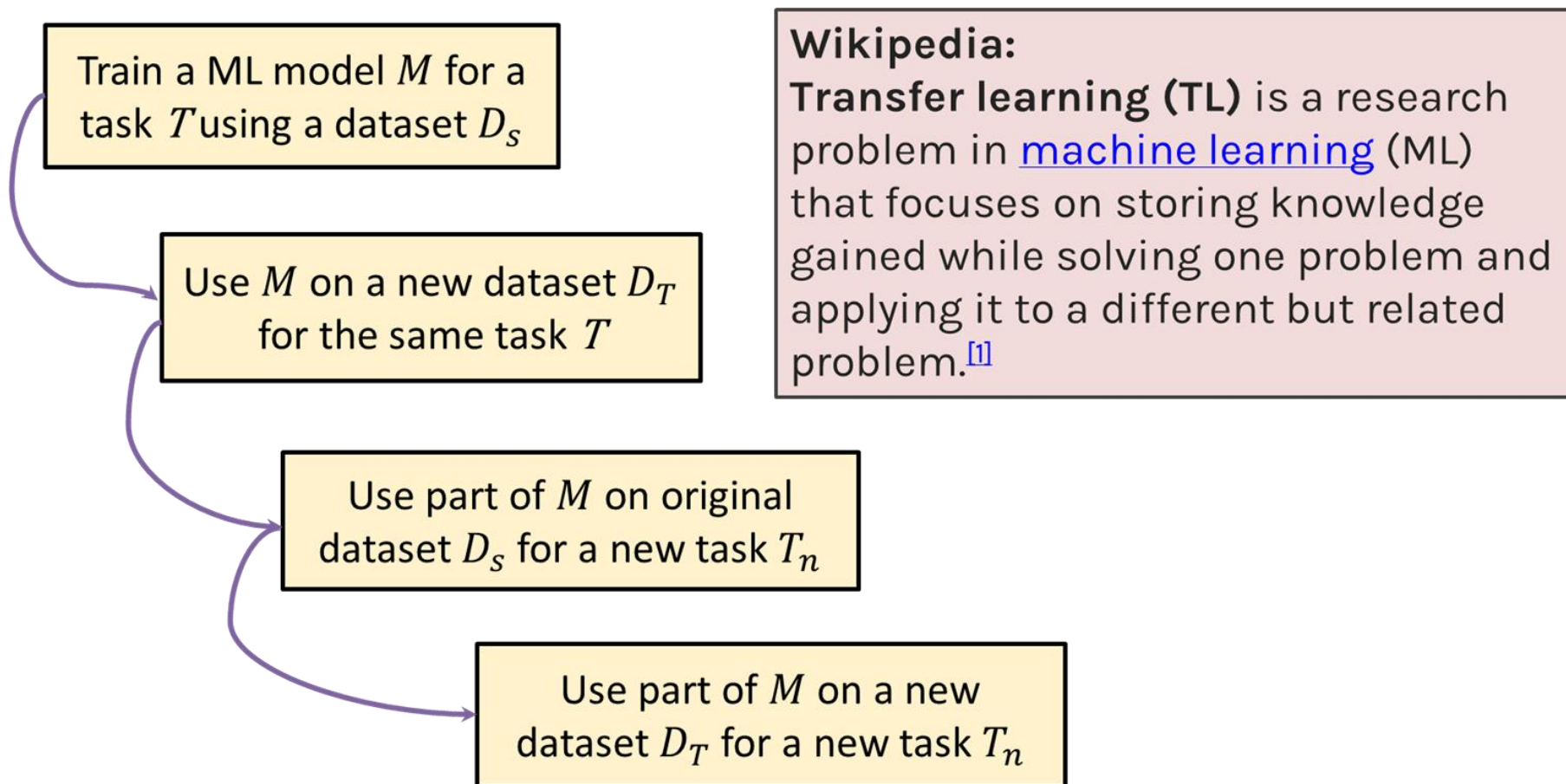
← Today

Image from Anil Jain, Google

Fully Built CNN (VGG)



How do you build an image classifier that can be trained in a few minutes on a CPU with very little data?



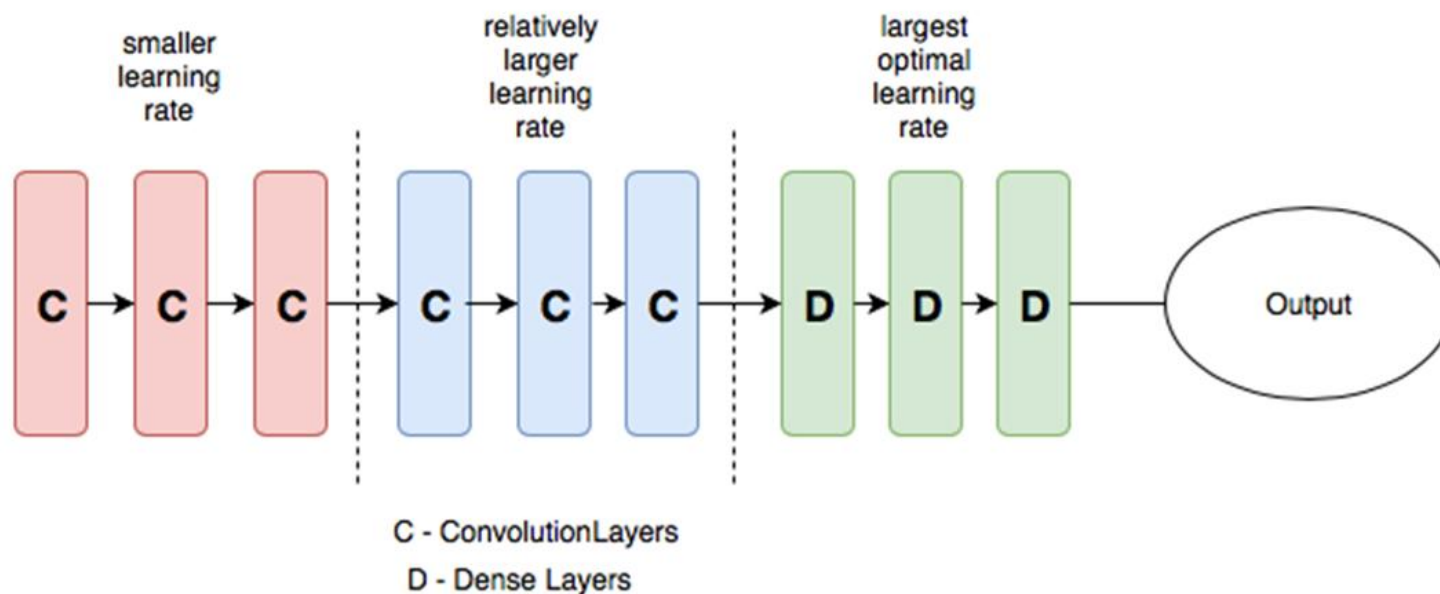
Recap – Convolutional Neural Networks (CNN)

A low learning rate can take a lot of time to train on the "later" layers. Since we trained the FC head earlier, we could probably retrain them at a higher learning rate.

General Idea: Train different layers at different rates.

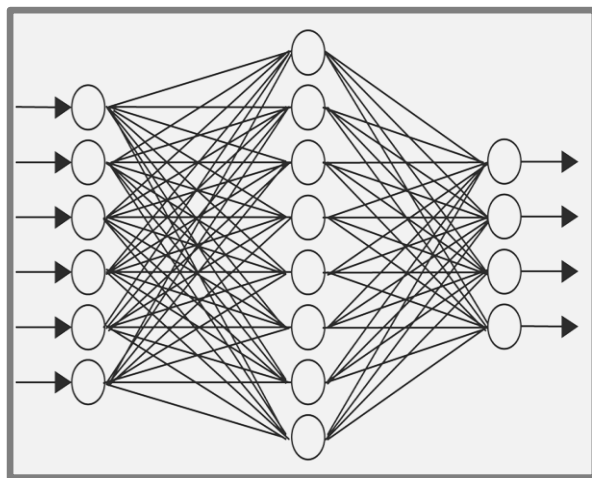
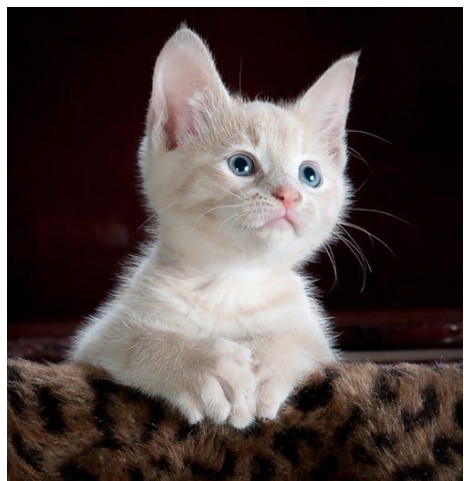
Each "earlier" layer or layer group (the color-coded layers in the image) can be trained at 3x-10x smaller learning rate than the next "later" one.

One could even train the entire network again this way until we overfit and then step back some epochs.



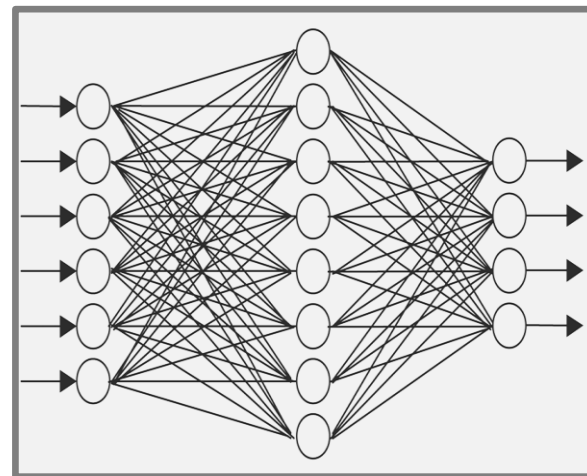
We train the two networks by minimizing the loss function
(cross entropy loss)

X



Feature Discovery Network

Features
 $\phi(x)$



Classification Network

Y



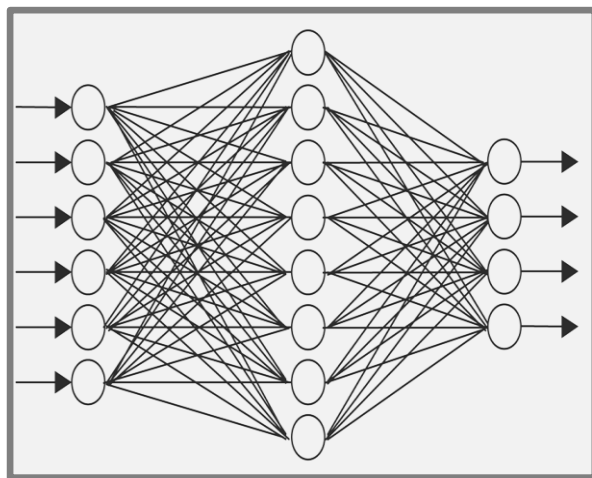
{Cat, Dog}

From Harvard CS109B

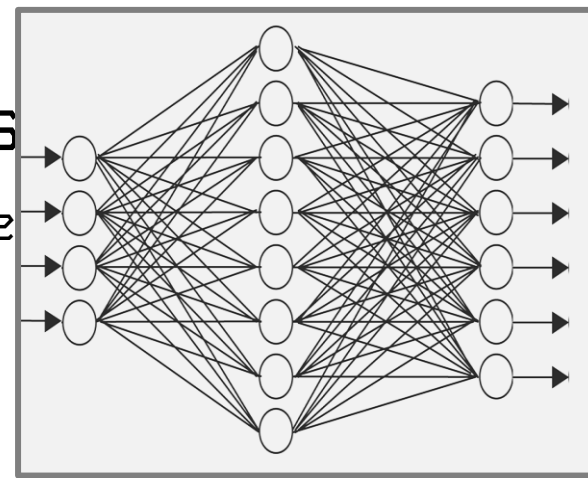
We train the two networks by minimizing the **reconstruction** loss function:

$$\mathcal{L} = \sum (x_i - \hat{x}_i)^2$$

AUTOENCODER

 x $\hat{x}$ 

Features
Latent space
 $\phi(x)$
 z



Feature Encoder Network

Decoder Network

This is an **autoencoder**. It gets that name because it automatically finds the best way to encode the input so that the decoded version is as close as possible to the input.

From Harvard CS109B

Brief history of encoding/decoding

Is this a new idea?

- MP3 can compress music files by a factor of 10 enabling digital storage and transmission large volumes of audio.
- JPG compresses images by a factor of 10-20 and enables storage and transmission of image data.
- These technologies led the way to the image-rich web and abundance of music that we enjoy today.

Brief history of encoding/decoding

We say that both MP3 and JPG take an input (a music or image file), and encode it into a compressed form.

Then we decode or decompress the intermediate version to some lower quality original version.

The greater the difference between the original version and the version post-decompression the greater the loss.

Example: Imagine you are in Boston and you want to write a birthday text to a special friend while walking home.

HANNAH HAPPY BIRTHDAY I LOVE YOU ROMAN

In the freezing Boston winter (-20C) you do not want to have your hands out in the open, so you shorten the message as much as possible using text-speak:

H HBD ILY R

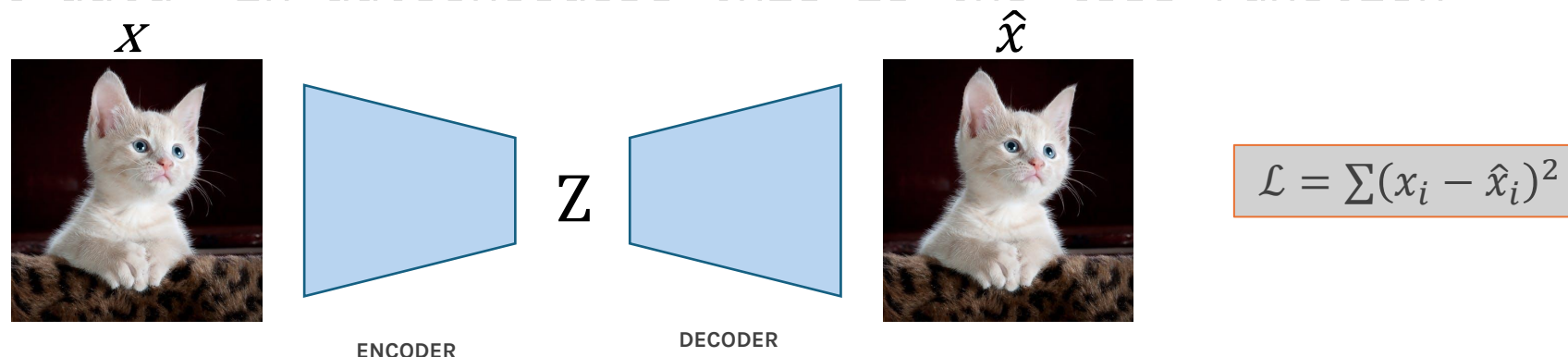
Your 36 characters message is compressed to 11 characters

Lossy and Lossless Encoding

Question: Is this an example lossy or lossless compression?

HPD is unambiguous given that it is her birthday today, but **R** could mean Roman or Raphael or Richard ... You can imagine the potential drama.

A way to test if a transformation is lossy or lossless is to consider if it can be inverted, or run backwards, to provide us with the original data. In autoencoders this is the loss function.



From Harvard CS109B

What are Autoencoders

A particular kind of learning architecture.

A mechanism of compressing inputs into a form that can later be decompressed, like the way MP3 compresses audio and JPG compresses images.

Autoencoders are more general than either MP3 or JPG.

They are usually used to ...

- reduce data dimensionality or find a more general representation for many tasks

- blend inputs from one input to another

- denoising, infilling

- ...

Use Cases

Data compression: Autoencoders naturally learn a compressed representation of input data.

Dimensionality reduction: The encodings learned by autoencoders can be used as input to larger, composite neural networks. Dimensionality reduction of complex data can extract features relevant to other tasks, as well as increase computational speed and efficiency

Anomaly detection and facial recognition: Autoencoders can detect anomalies, fraud or other defects—and, conversely, confirm a genuine match—by determining the reconstruction loss of examined data relative to the “normal” or “genuine” example it’s compared against.

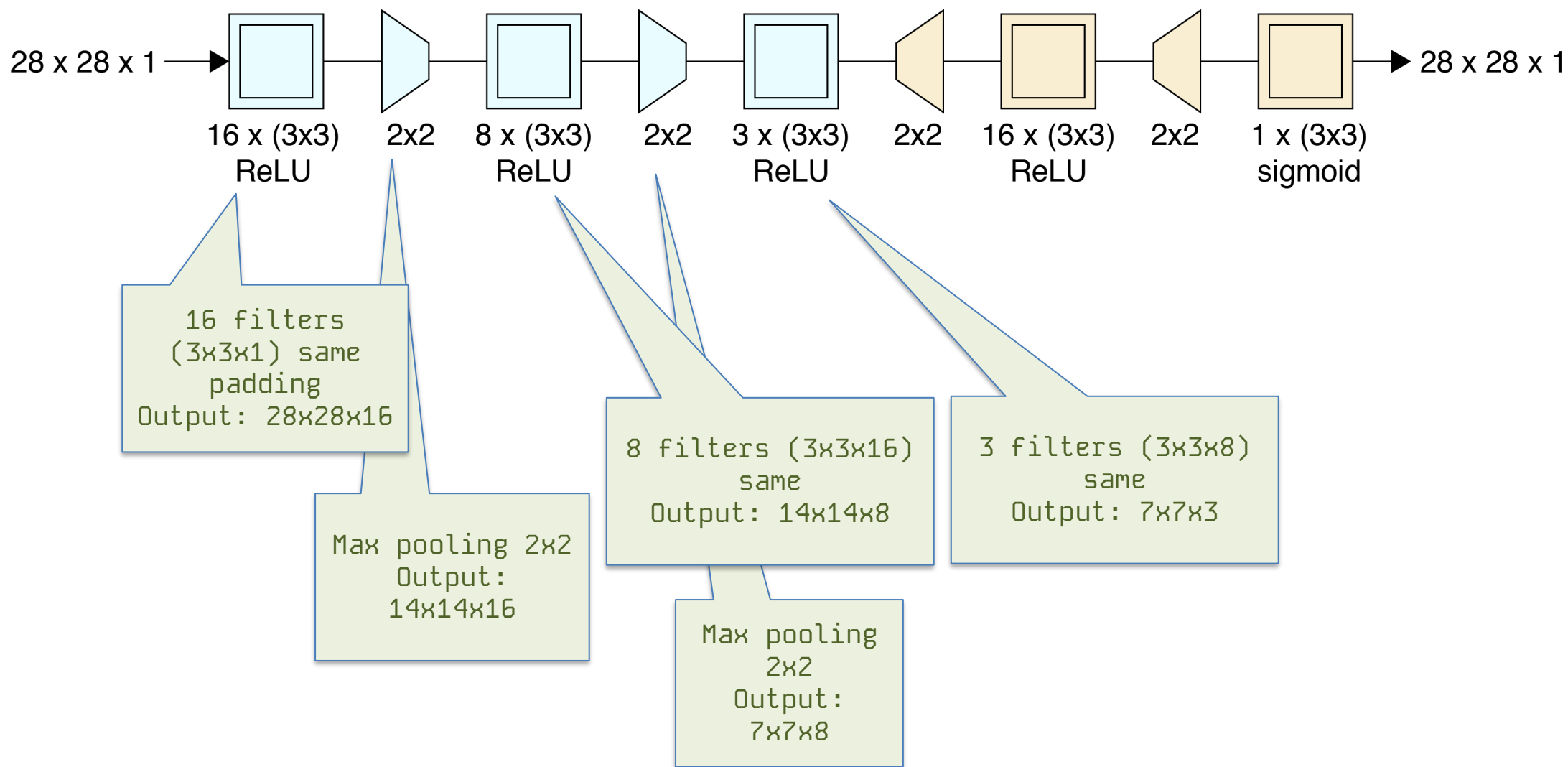
Image denoising and audio denoising: Denoising autoencoders can remove extraneous artifacts or corruption that does not match the latent space representation learned by the network.

Image reconstruction: Inverting techniques learned for denoising, autoencoders can fill in missing elements of an image. They can be similarly used to colorize images.

Generative tasks: VAEs and adversarial autoencoders (AAEs), which learn probabilistic distributions akin to those learned by VAEs but use an adversarial discriminator network in place of KL divergence, have been used with great success for generative tasks. Prominent generative applications of autoencoders include OpenAI’s original DALL-E model for image generation and the generation of molecular structures used in medications.

From IBM

Convolutional Autoencoders

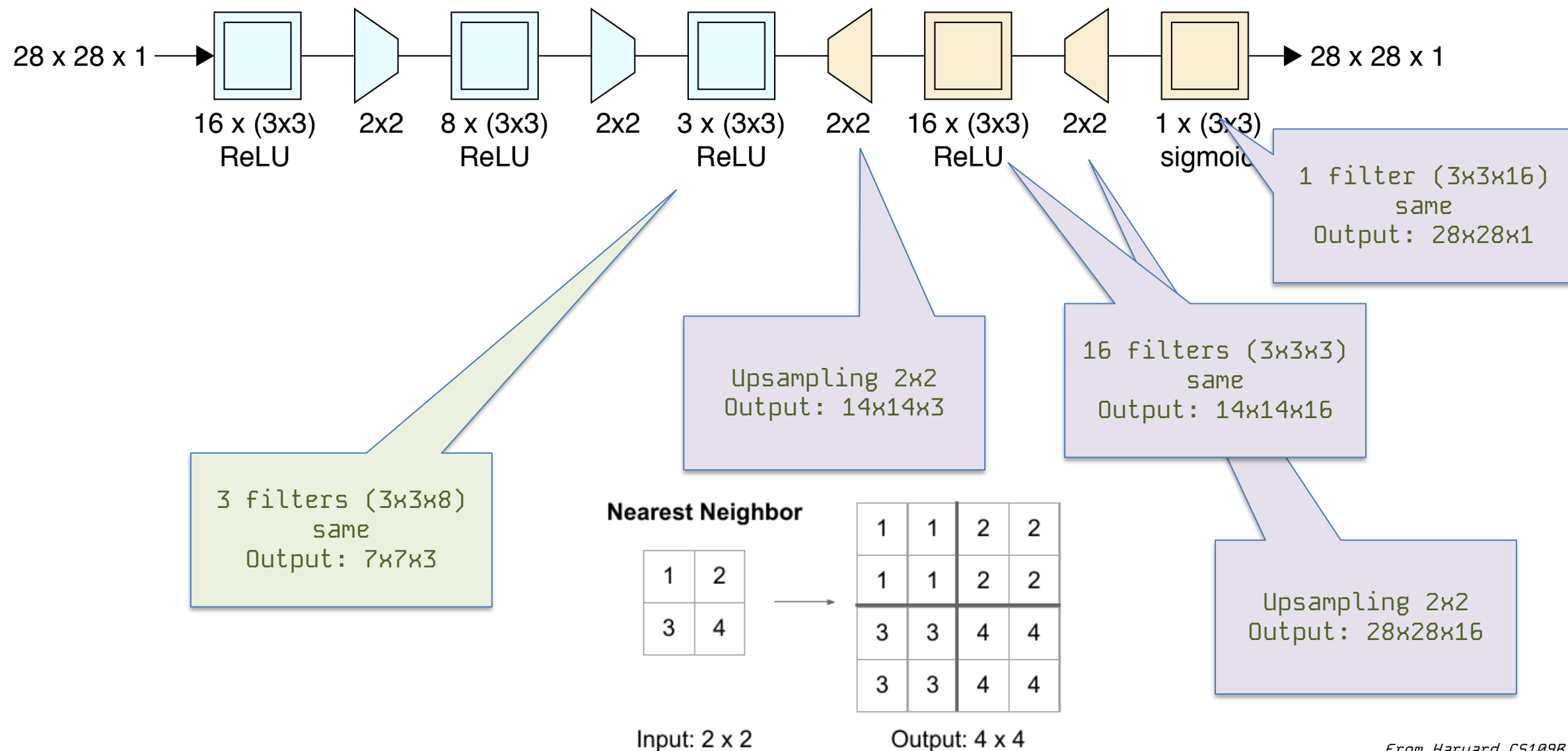


From Harvard CS109B

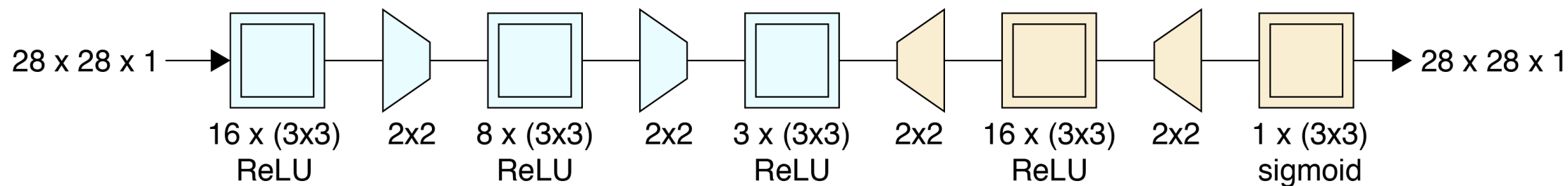
SUPERVISED 8

SUPERVISED 9

Convolutional Autoencoders



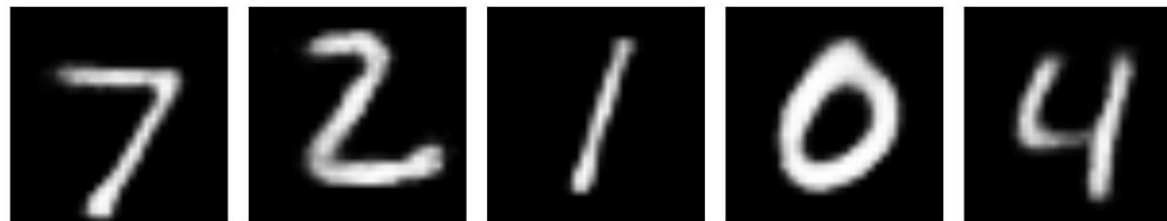
From Harvard CS109B



Original Images



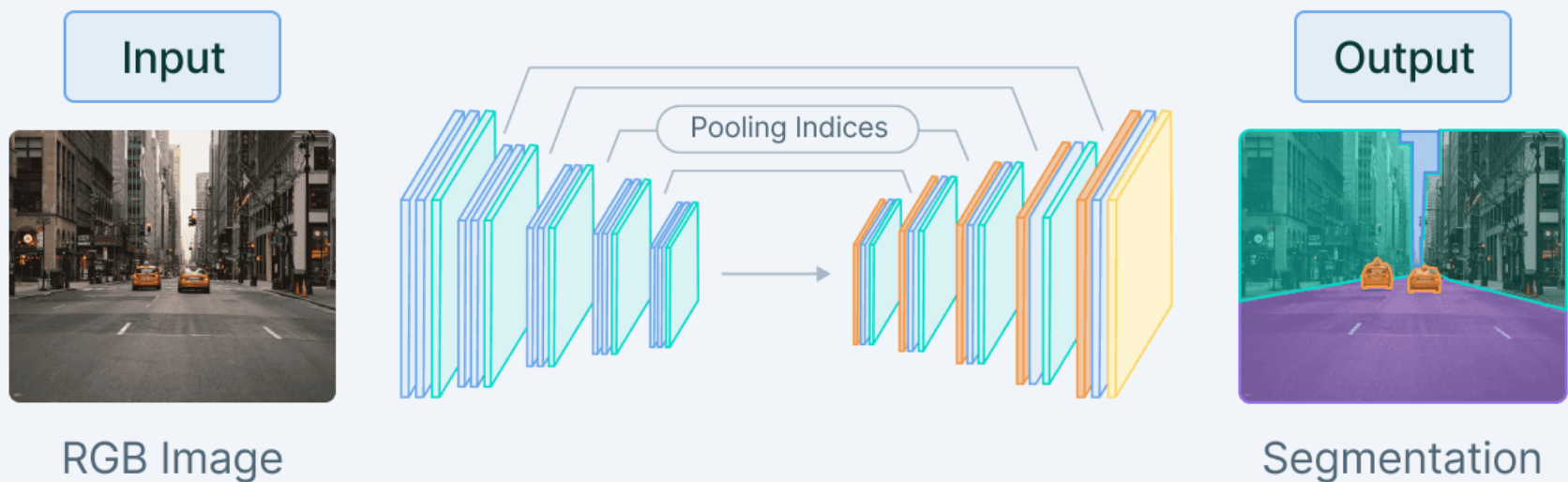
Reconstructed Images
with Conv AE



ConvAE latent variables
DeepFCN latent variables

From Harvard CS109B

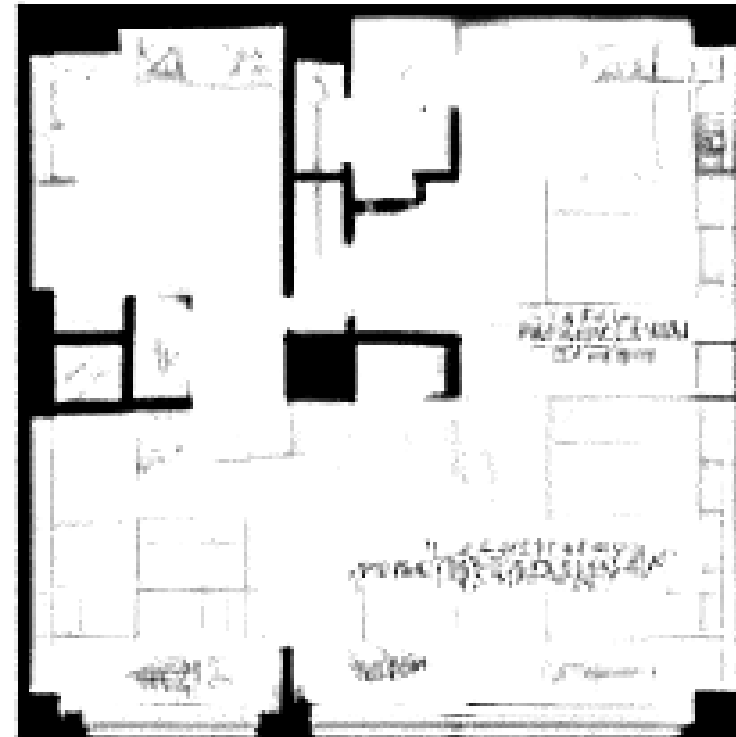
Convolutional encoder-decoder



From Harvard CS109B

Google Colab
<https://bit.ly/BPS5231-L9>

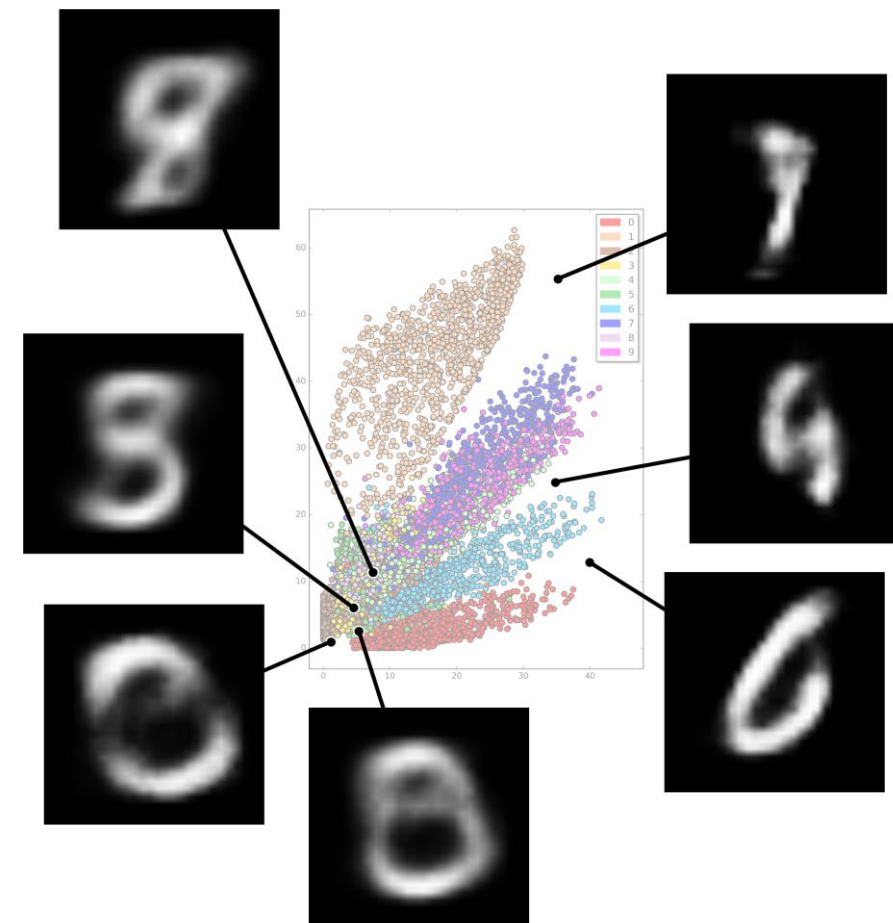
Another use of generating new data is to give us ideas and options. Suppose we're planning a house. We can give the computer the space we have available, and its location. From this, the computer can give us some ideas.



From Harvard CS109B

Problems with Autoencoders

- Gaps in the latent space
- Discrete latent space
- Separability in the latent space



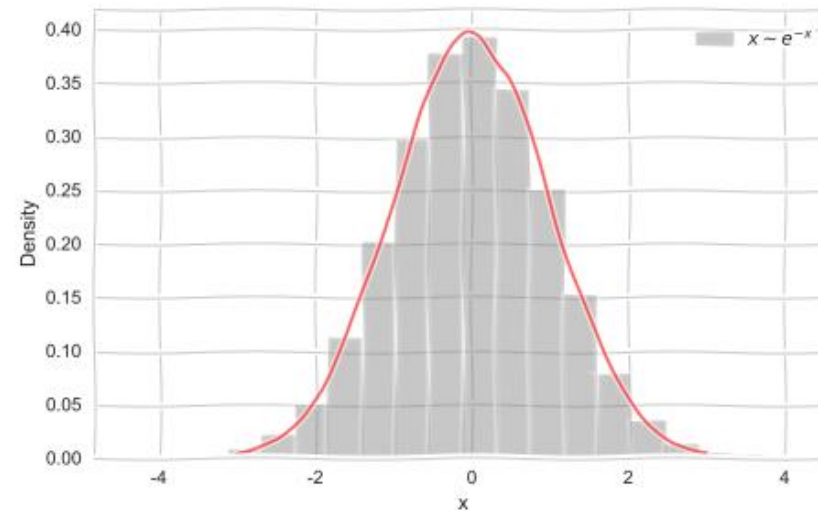
From Harvard CS109B

Imagine we want to generate data from a distribution,

$$x \sim p(x)$$

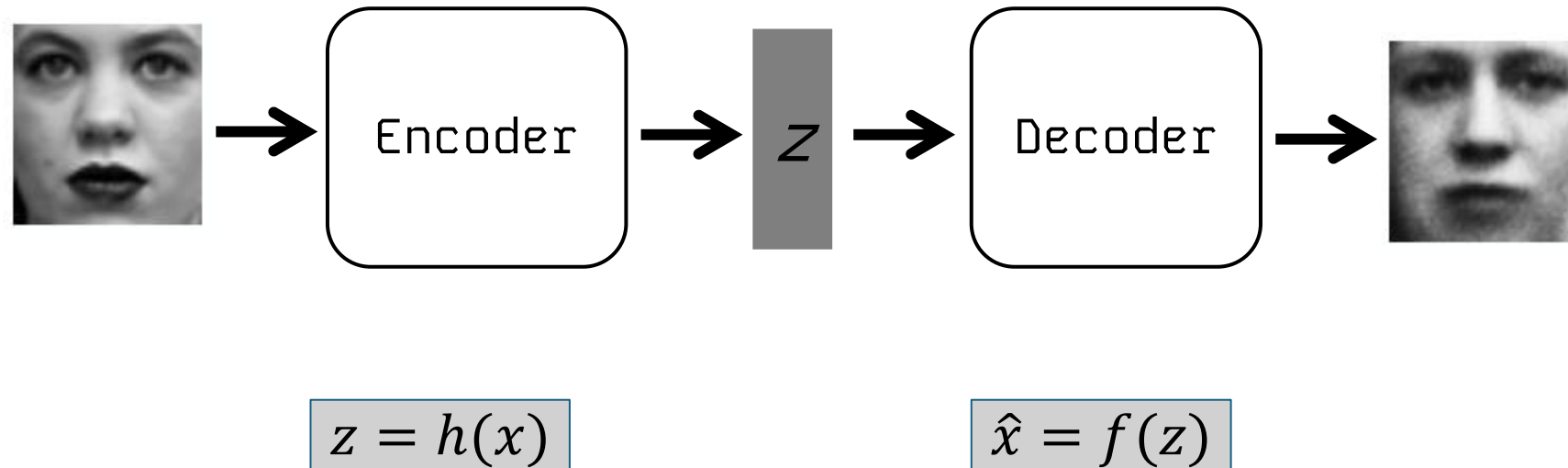
e.g.

$$x \sim \mathcal{N}(\mu, \sigma)$$



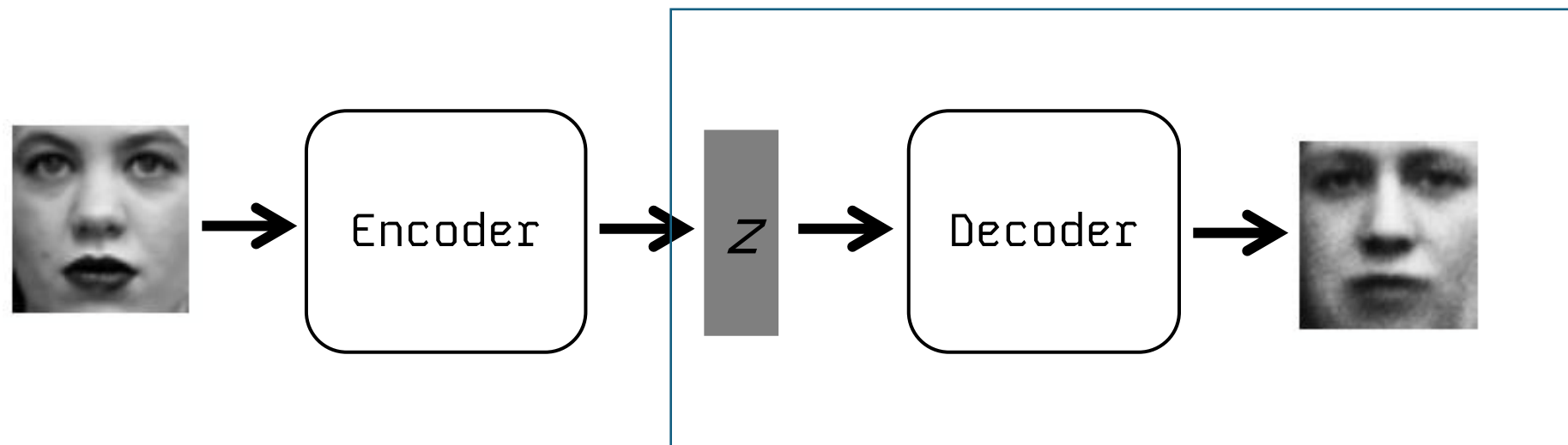
From Harvard CS109B

In traditional autoencoders, we can think of encoder and decoders as some function mapping.



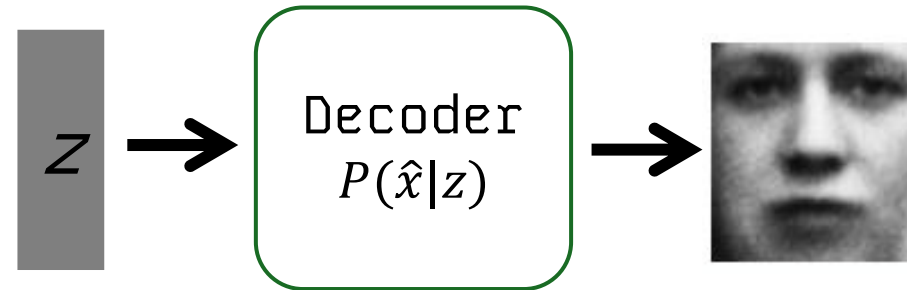
From Harvard CS109B

To go to variational autoencoders, we need to first add some stochasticity and think of it as a probabilistic modeling.



From Harvard CS109B

Sample from $g(z)$
*e.g. Standard
Gaussian*



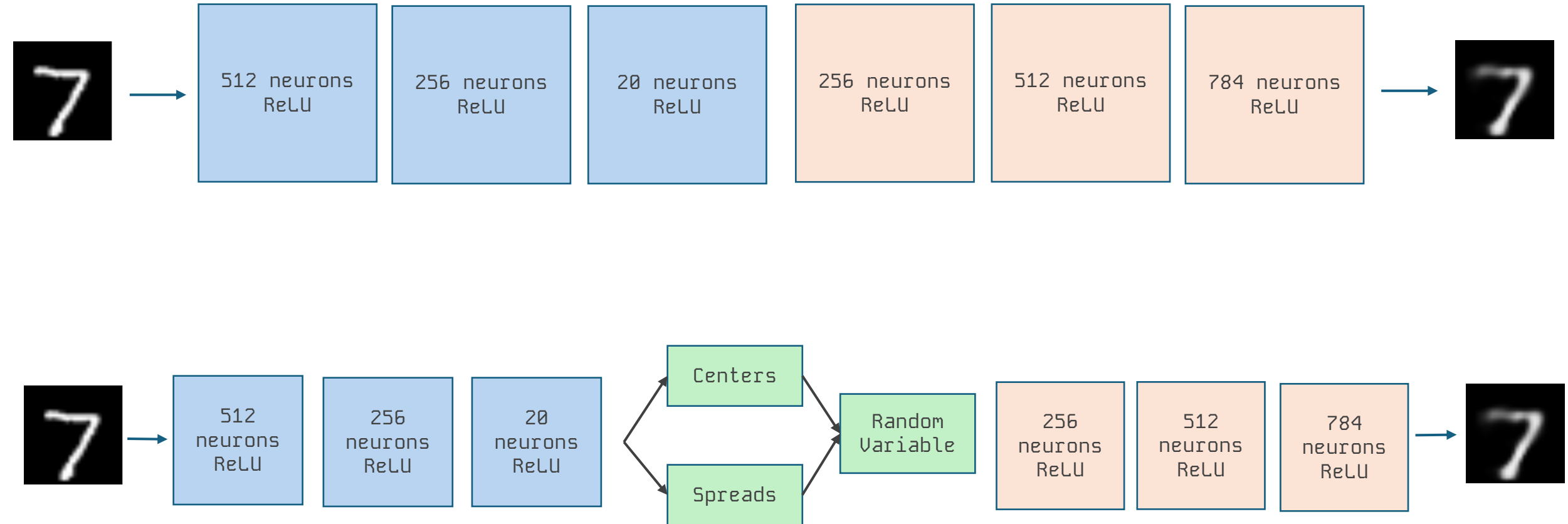
$$z \sim g(z)$$

$$\hat{x} = f(z)$$

$$\hat{x} \sim P(x|z)$$

From Harvard CS109B

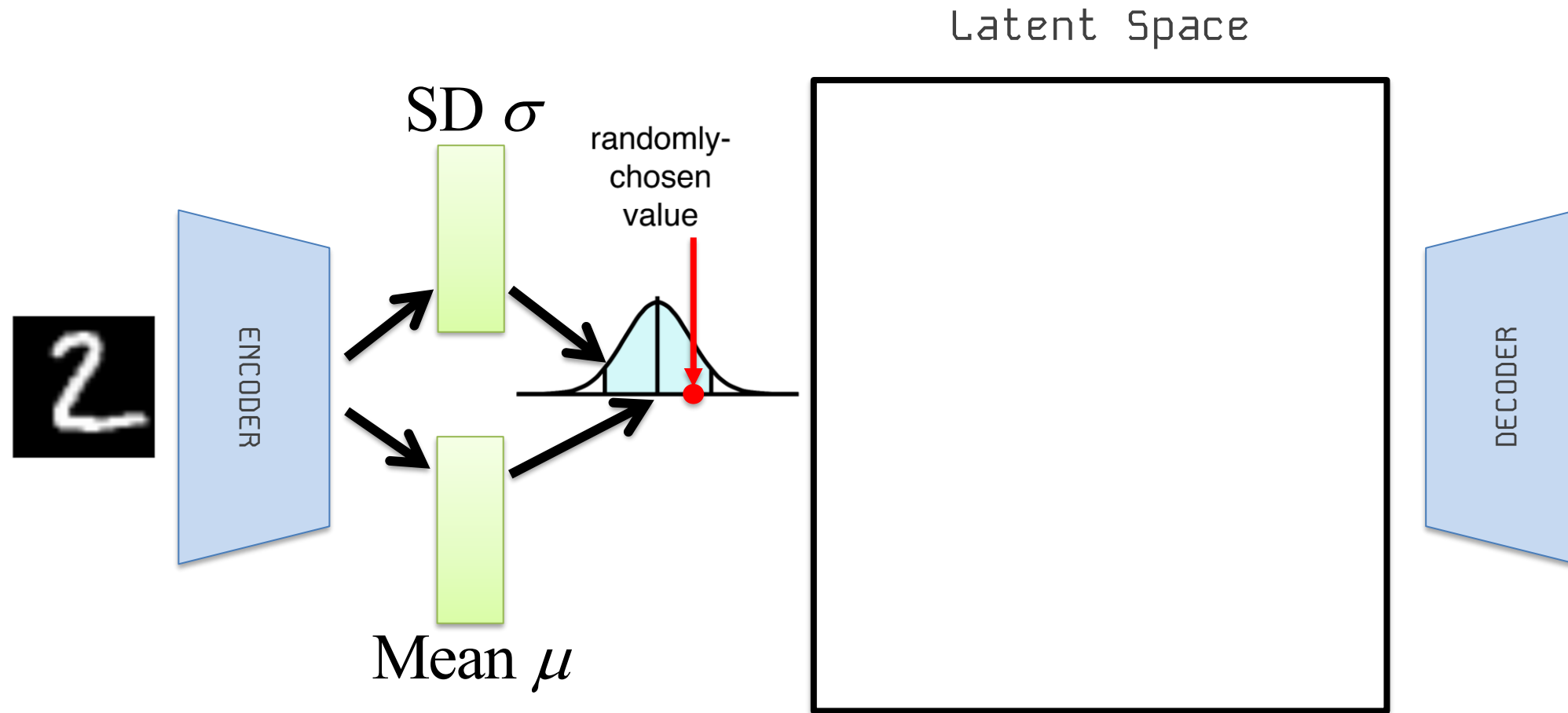
Variational Autoencoder



From Harvard CS109B

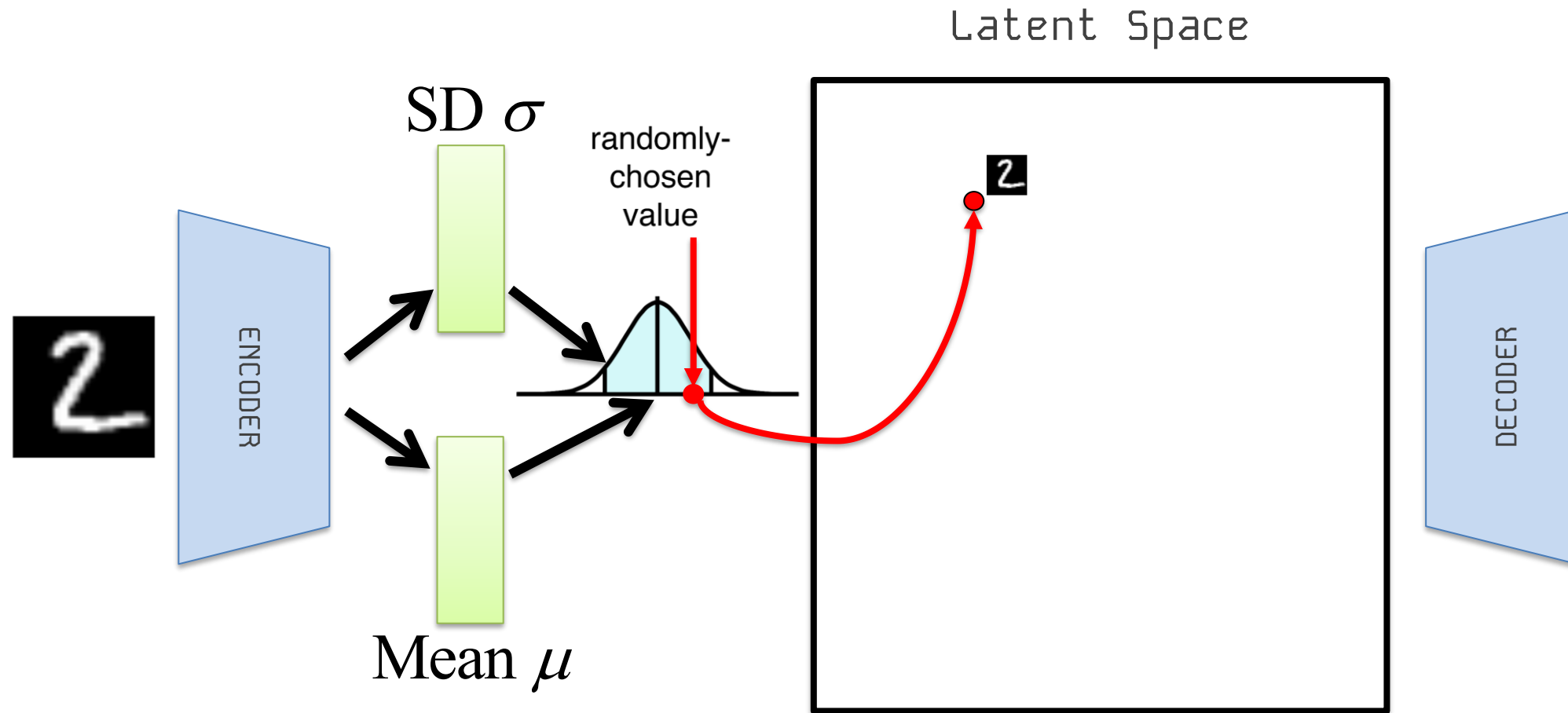
SUPERVISED 8

SUPERVISED 9



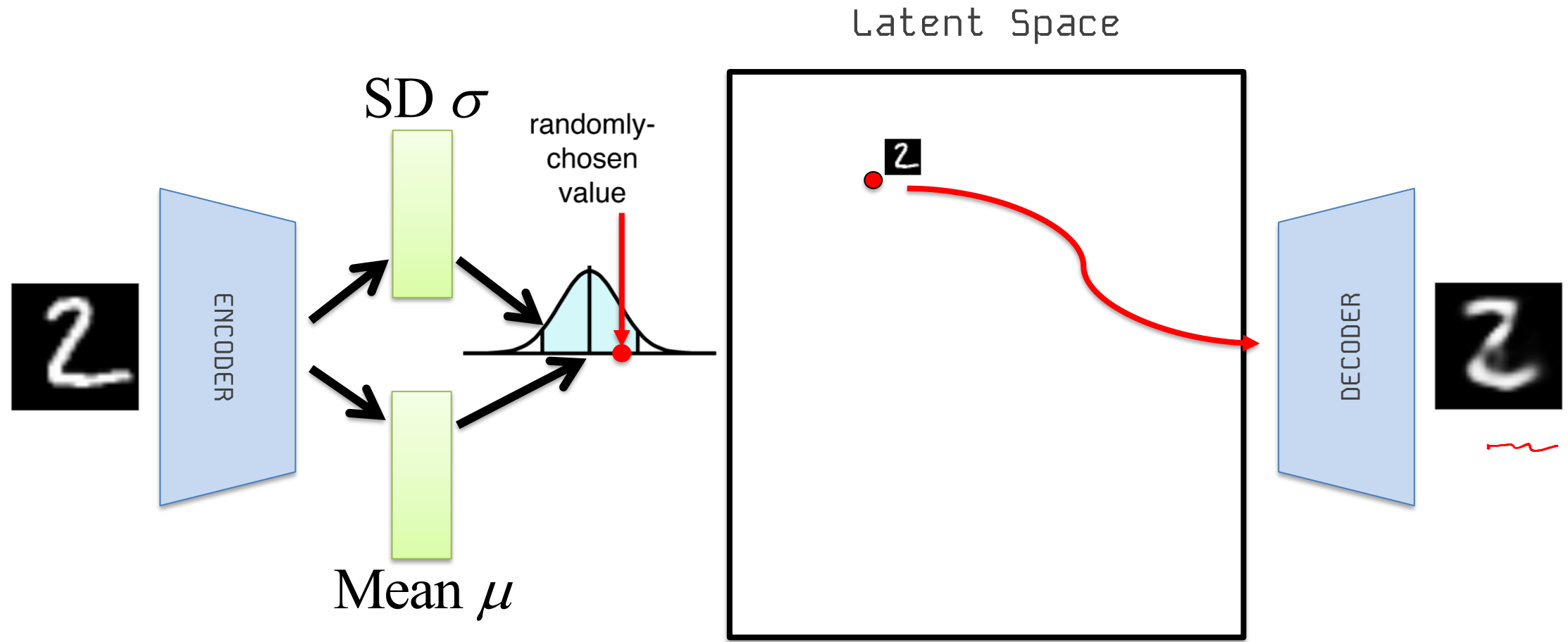
Encode the first sample (a “2”) and find μ_1, σ_1

From Harvard CS109B



Sample $z_1 \sim N(\mu_1, \sigma_1)$

From Harvard CS109B

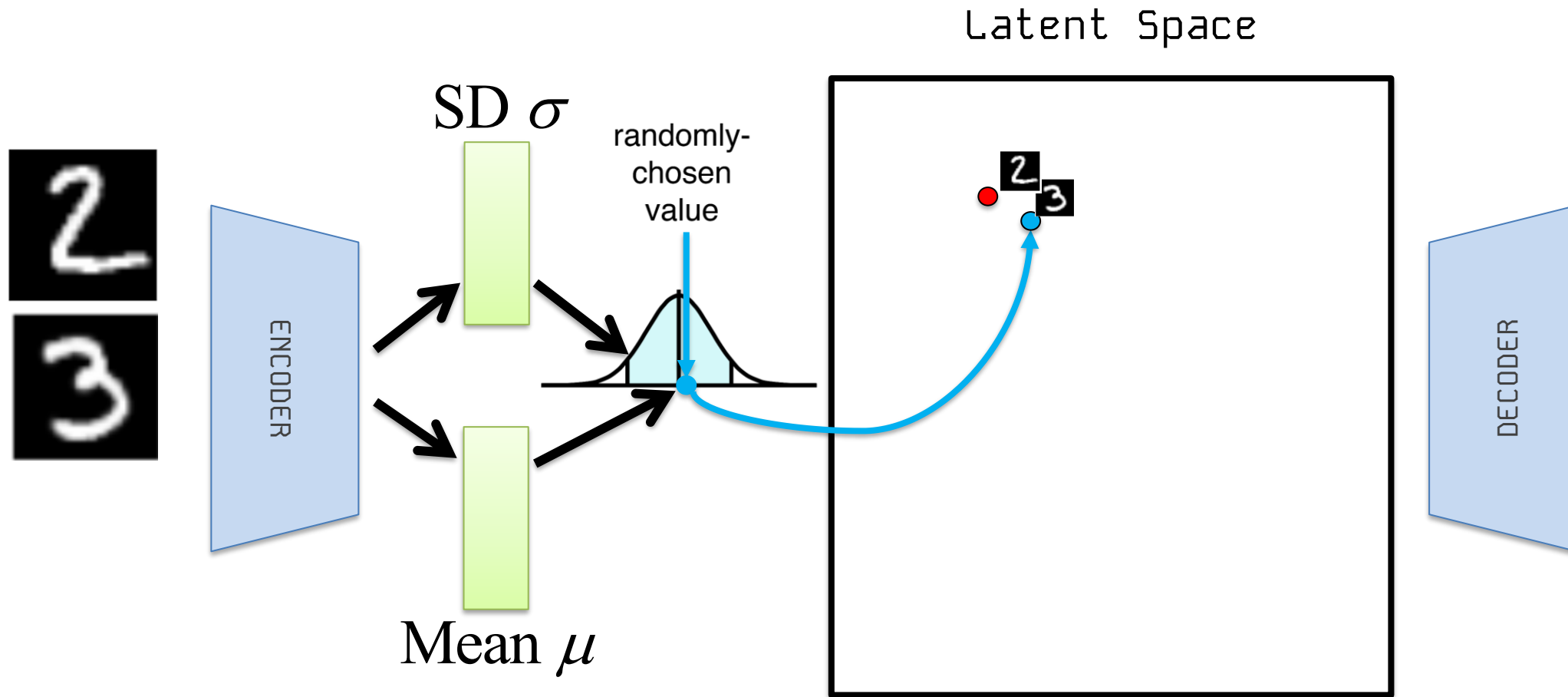


Decode to $\hat{x}_1$

From Harvard CS109B

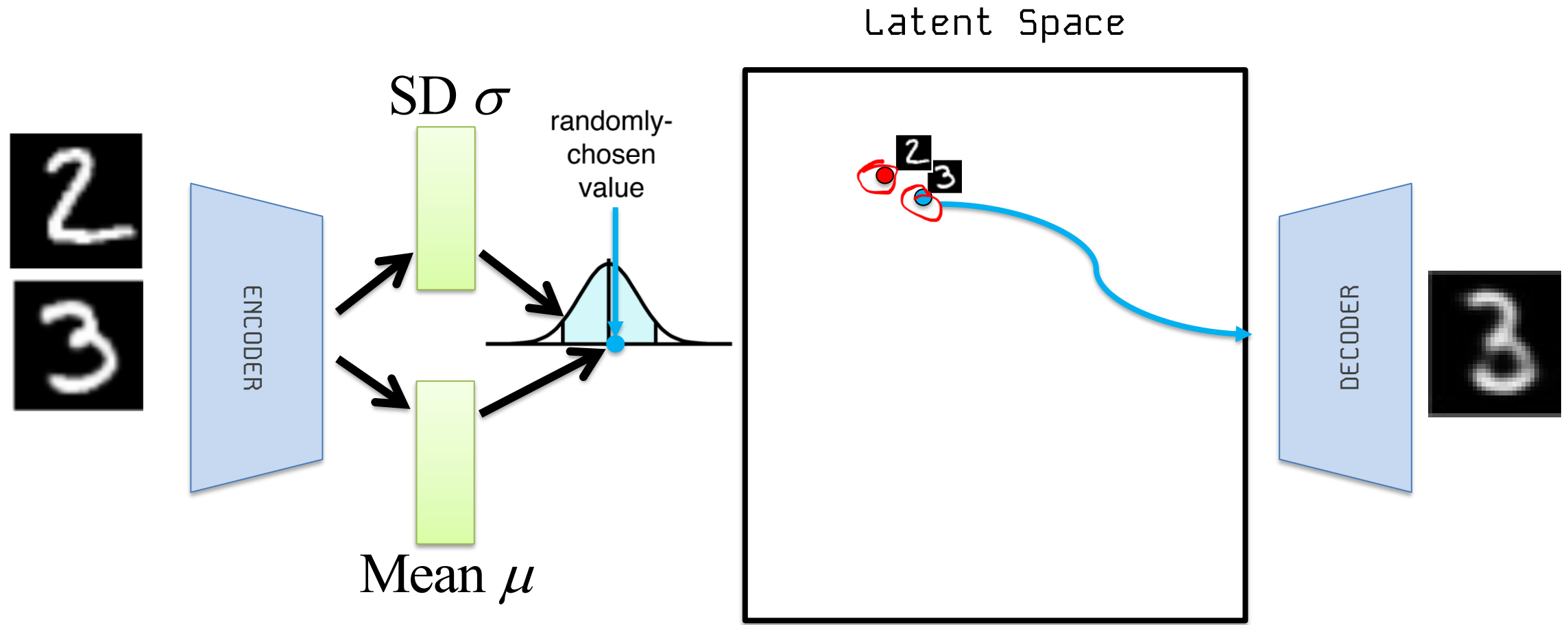
SUPERVISED 8

SUPERVISED 9



Encode the second sample (a “3”) find μ_2, σ_2 . Sample $z_2 \sim N(\mu_2, \sigma_2)$

From Harvard CS109B

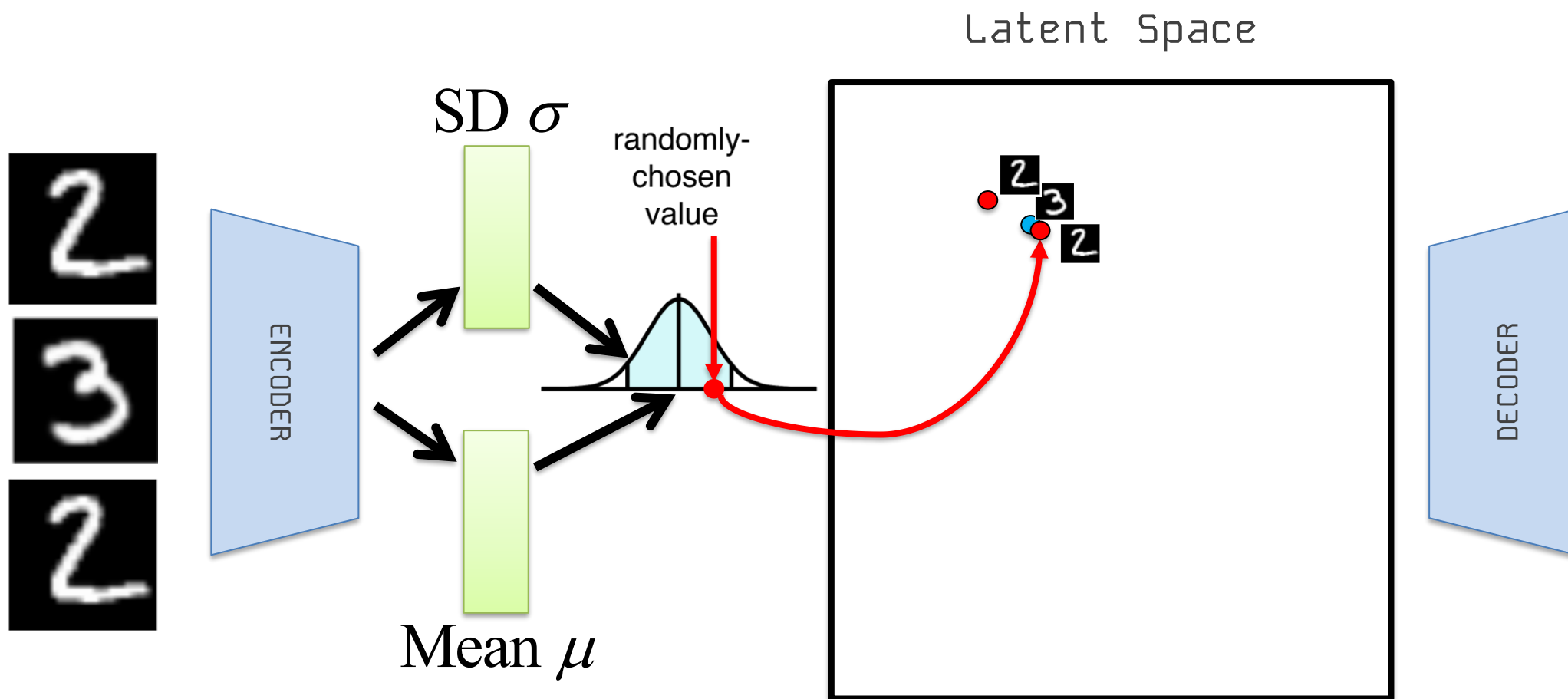


Decode to $\hat{x}_2$

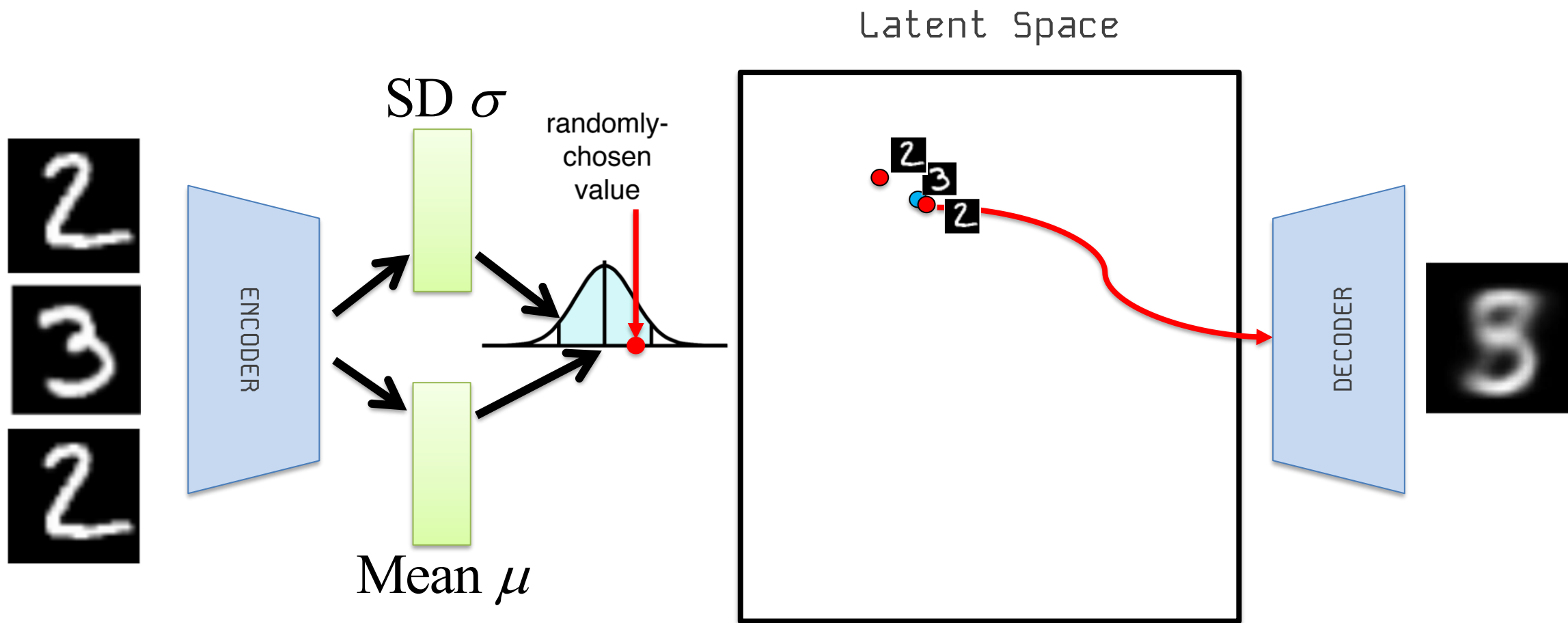
From Harvard CS109B

SUPERVISED 8

SUPERVISED 9



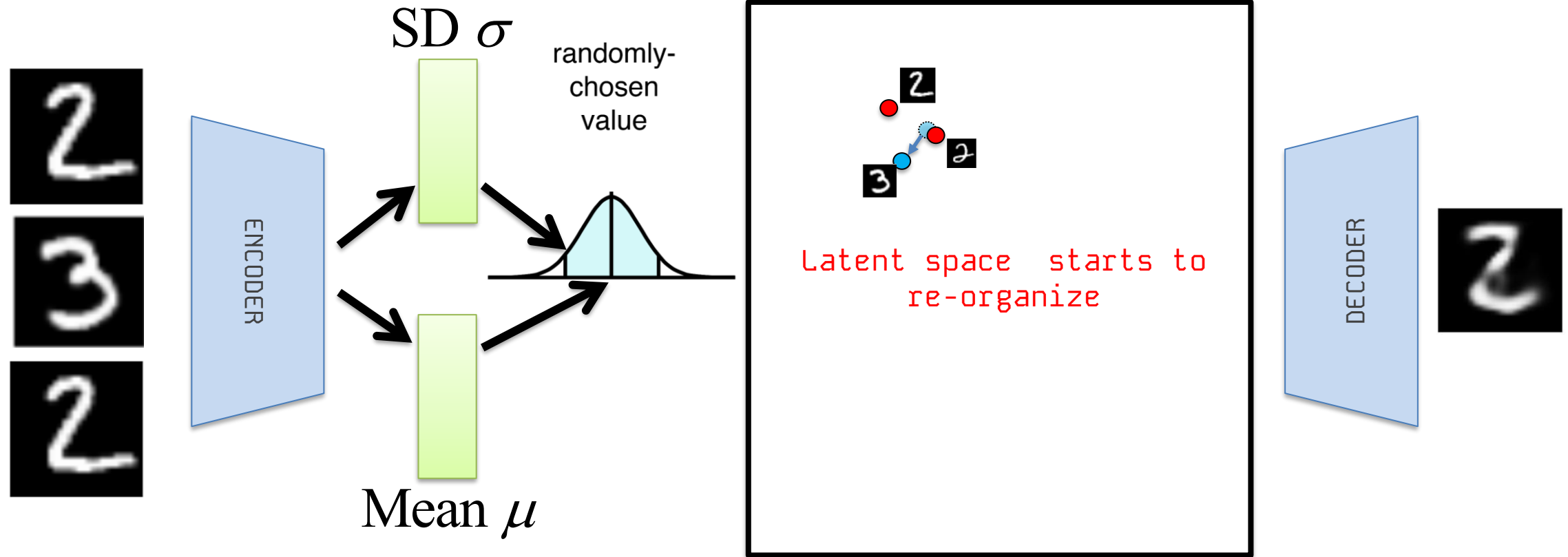
Train with the first sample (a “2”) again and find μ_1, σ_1 . However $z_1 \sim N(\mu_1, \sigma_1)$ will not be the same. *It can happen to be close to the “3” in latent space.* From Harvard CS109B



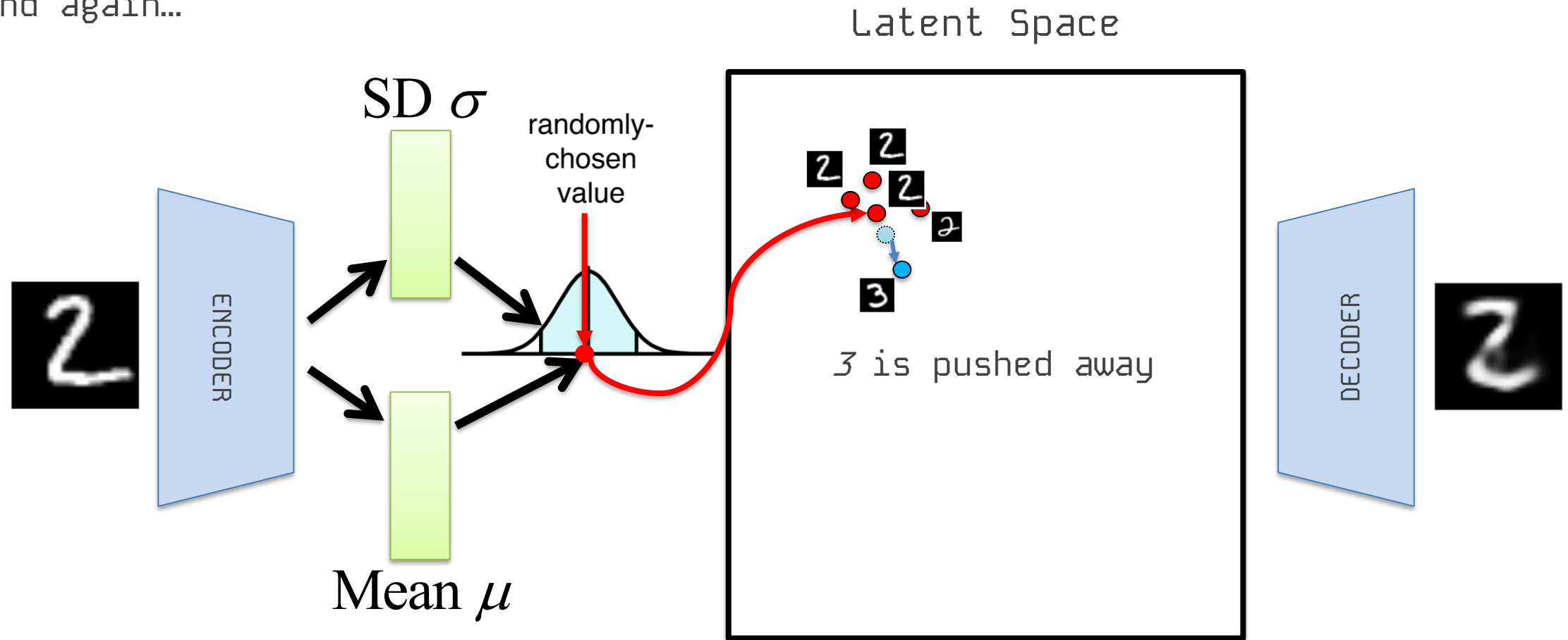
Decode to $\hat{x}_1$. Since the decoder only knows how to map from latent space to $\hat{x}$ space, it will return a "3".

From Harvard CS109B

Train with 1st sample again

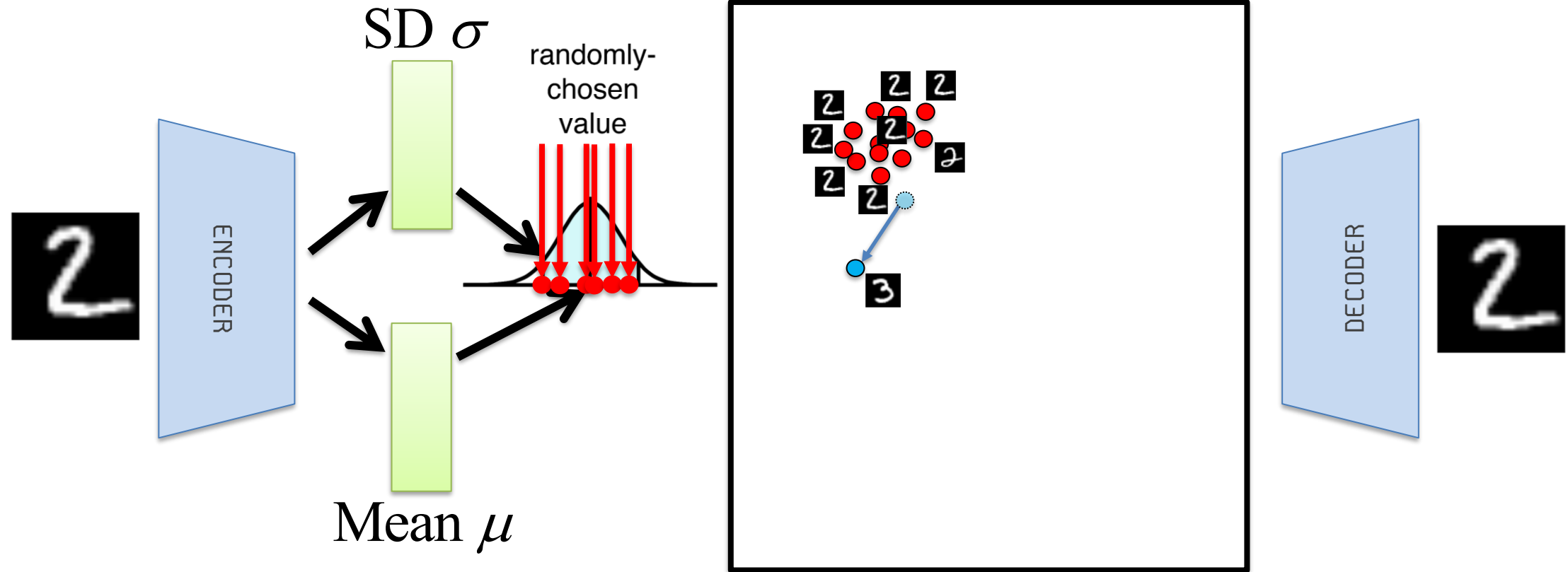


And again...



From Harvard CS109B

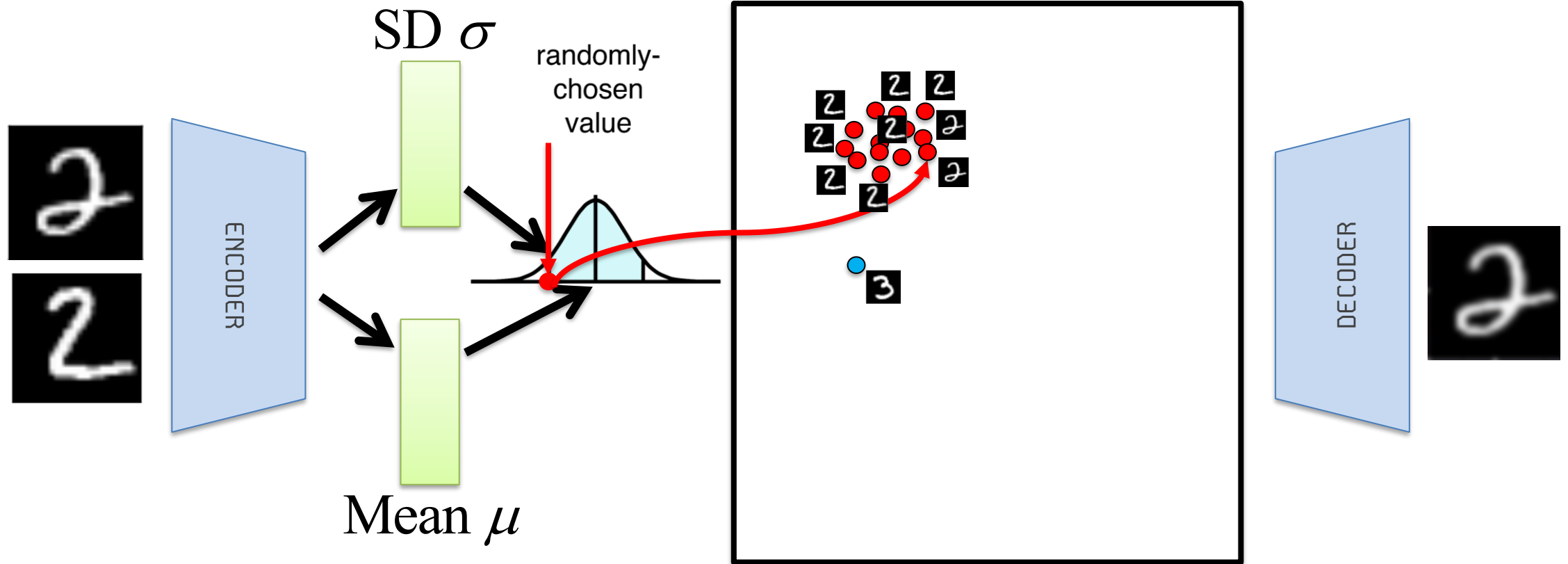
Many times...



From Harvard CS109B

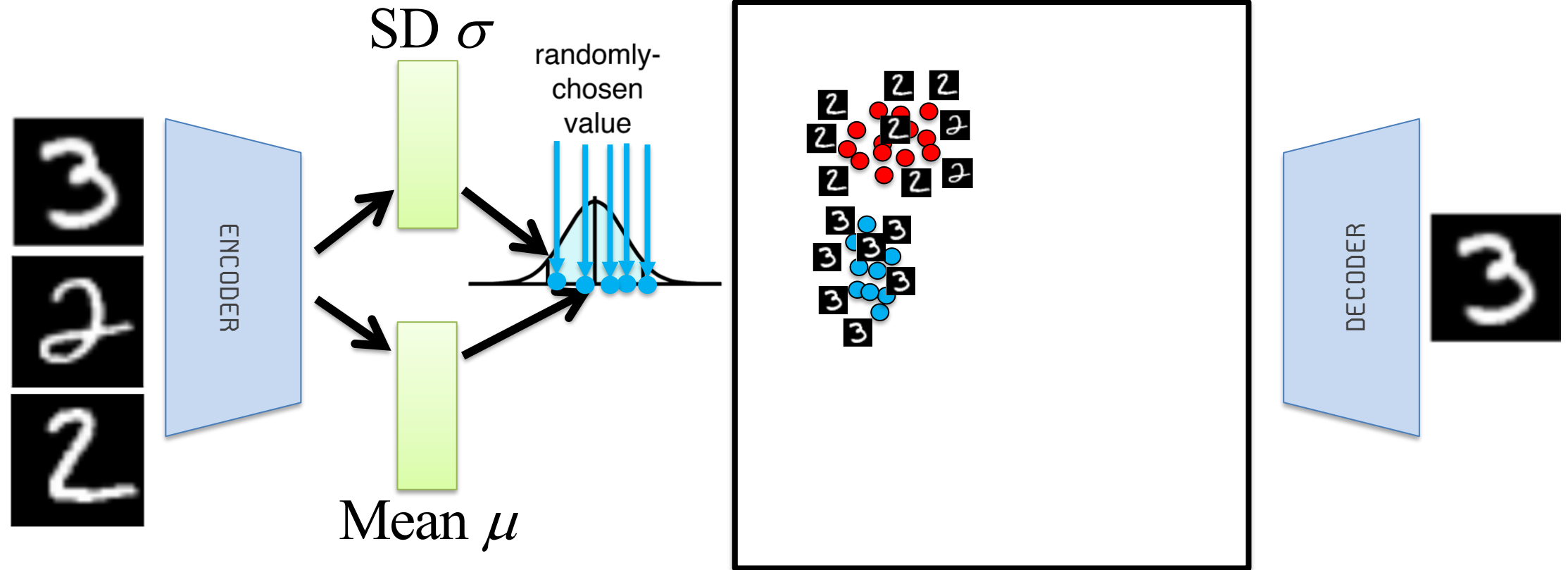
Now lets test again

Latent Space



From Harvard CS109B

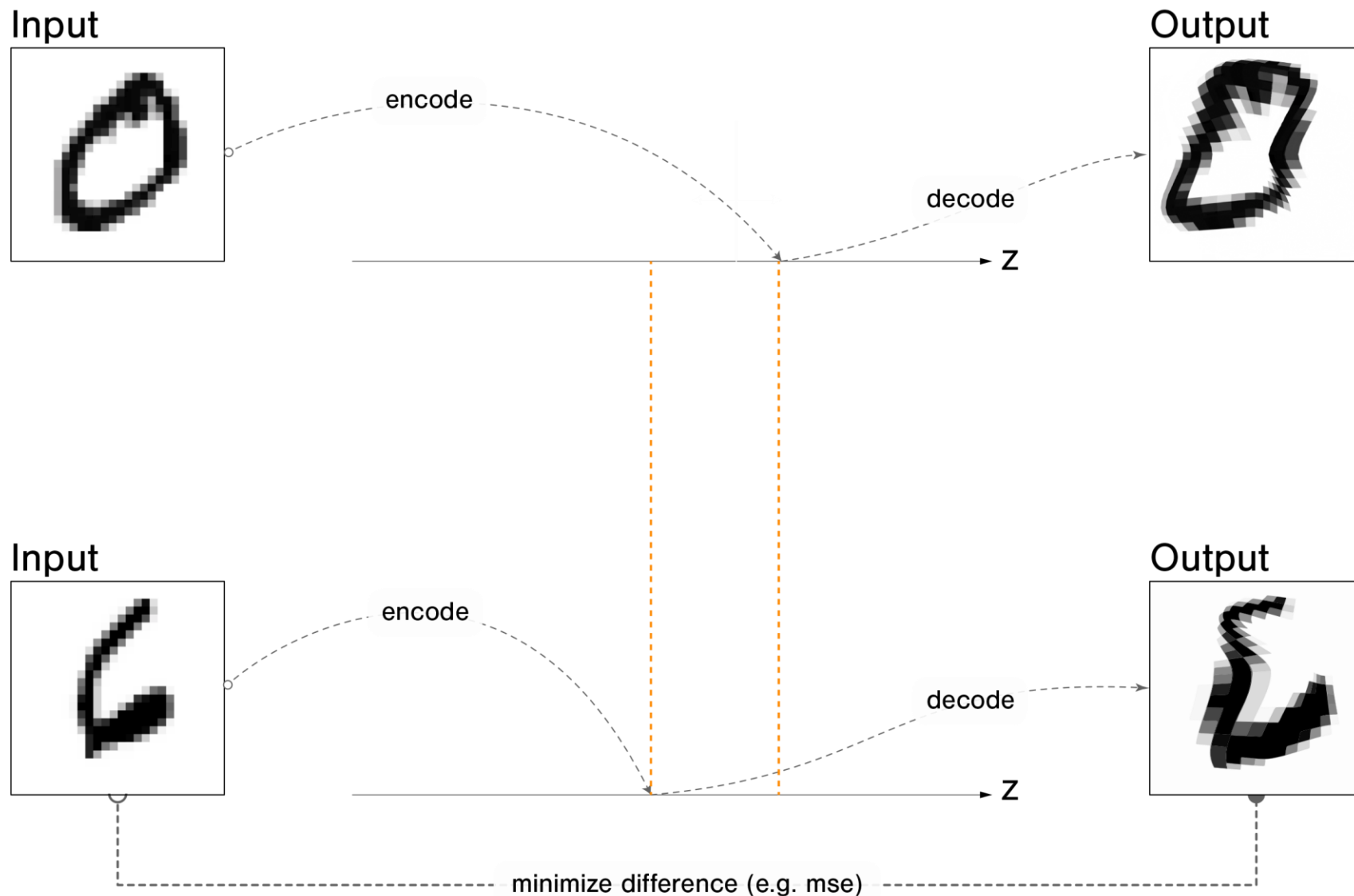
Many times...



From Harvard CS109B

Traditional Autoencoder

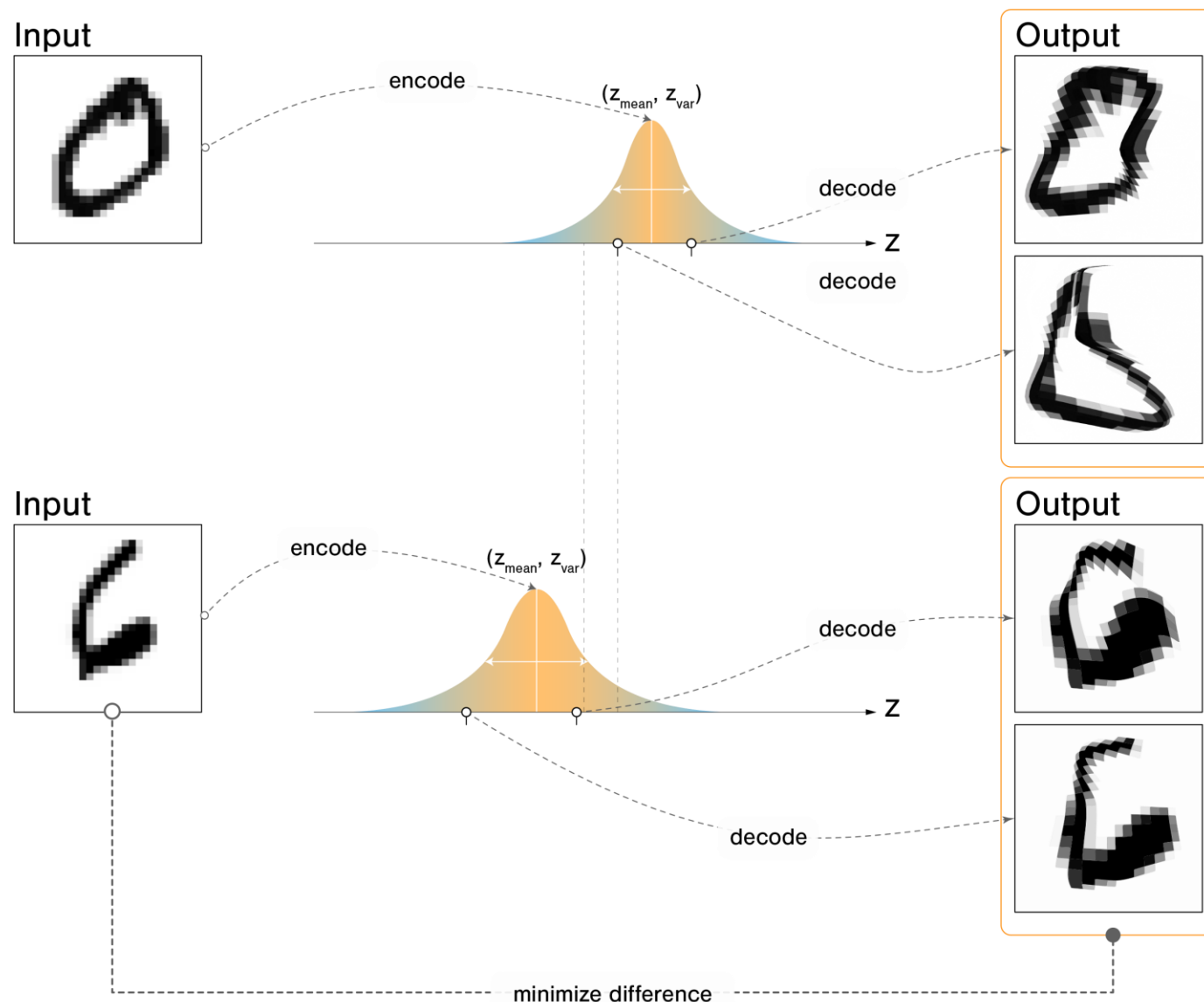
- Map input to latent space (*encode*)
- Reconstruct from point in latent space (*decode*)
- Minimize the difference



Variational Autoencoder

- Map input to probability distribution over latent space (*encode*)
- Sample point from distribution in latent space and reconstruct (*decode*)
- Minimize the difference (more sophisticated than mse)

Enforces strong continuity in latent space



L09.1

Supervised 8

Autoencoders

Variational Autoencoders

L09.2

Supervised 9

Generative Adversarial Networks

Adversaries



SUPERVISED 8

SUPERVISED 9

Adversaries

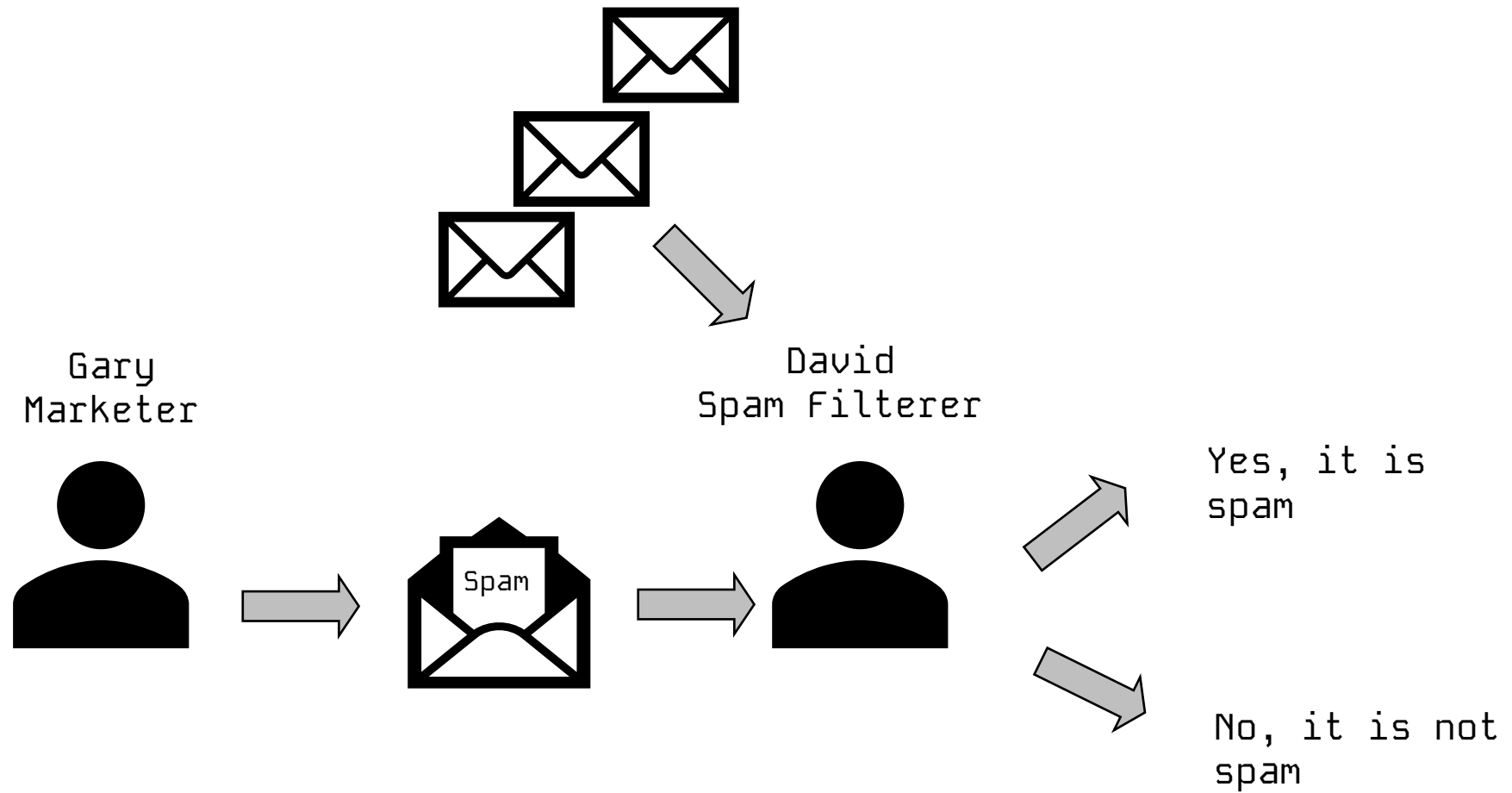


Adversaries



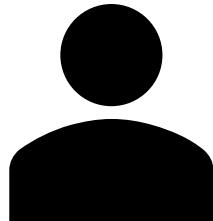
SUPERVISED 8

SUPERVISED 9



From Harvard CS109B

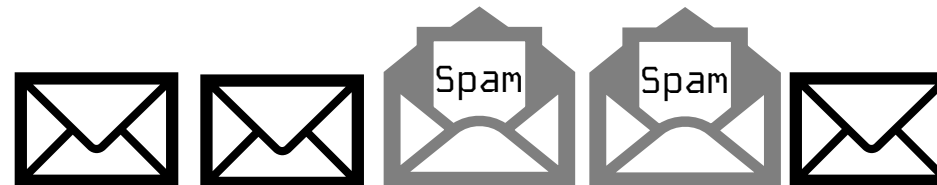
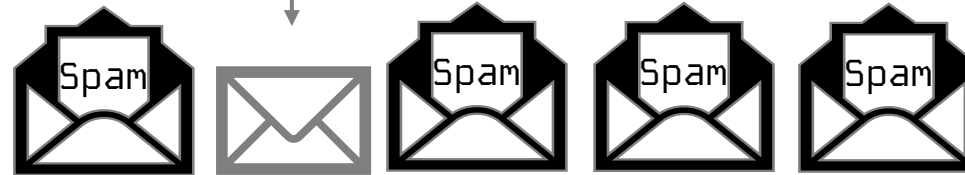
David
*checks the
results*



Yes, it is
spam

No, it is
not spam

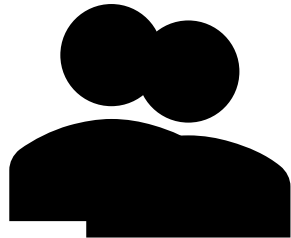
Discarded a valid email

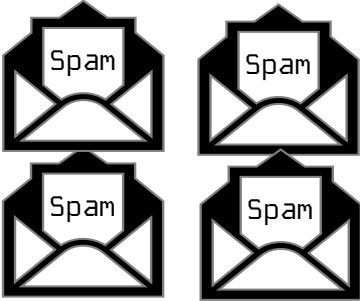
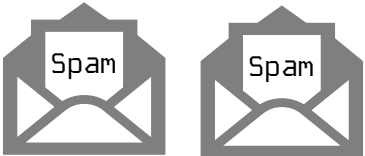
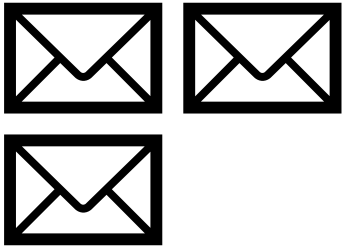


Allow some spams

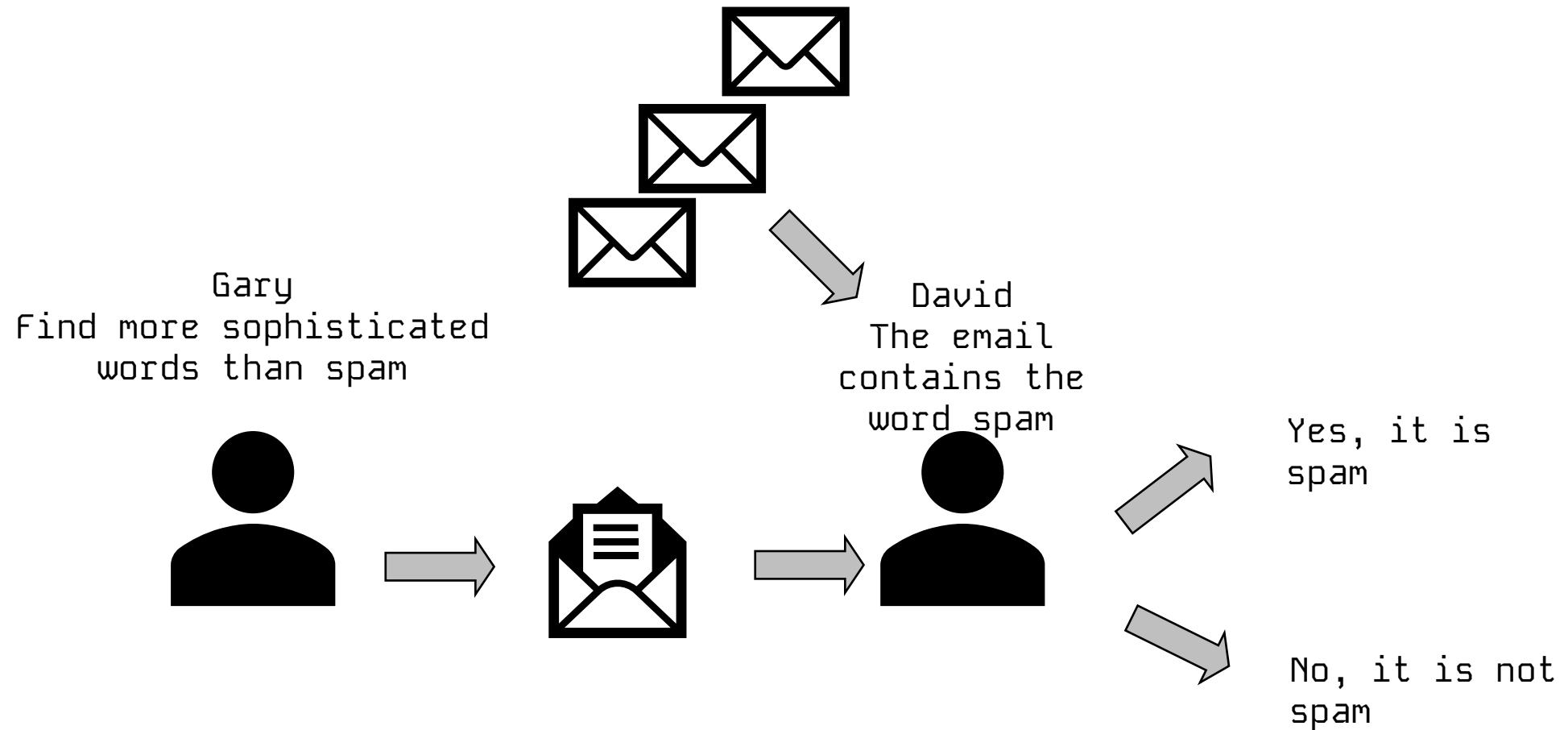
From Harvard CS109B

David and Gary
*learned what went
wrong*

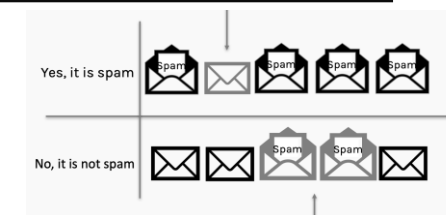


	It was spam, for real	It was not spam
Yes, it is spam		
No, it is not spam		

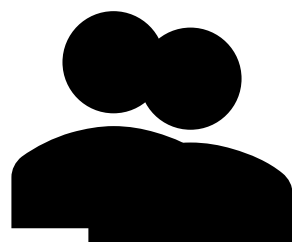
From Harvard CS109B



From Harvard CS109B



David and Gary
*learned what
went wrong*



Yes, it is
spam



Discarded a valid email

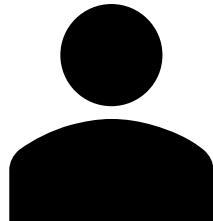
No, it is not
spam

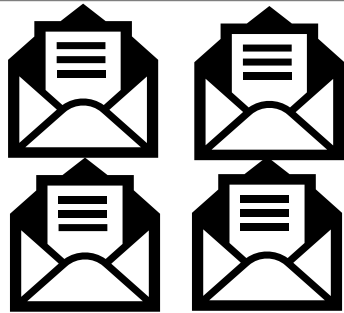
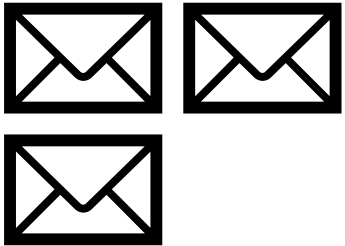


Allow fewer spams

From Harvard CS109B

David
*learned what
went wrong*

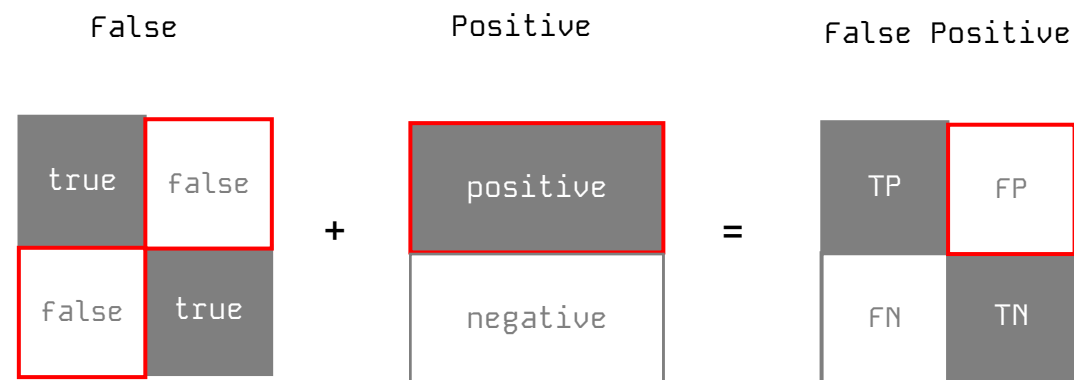
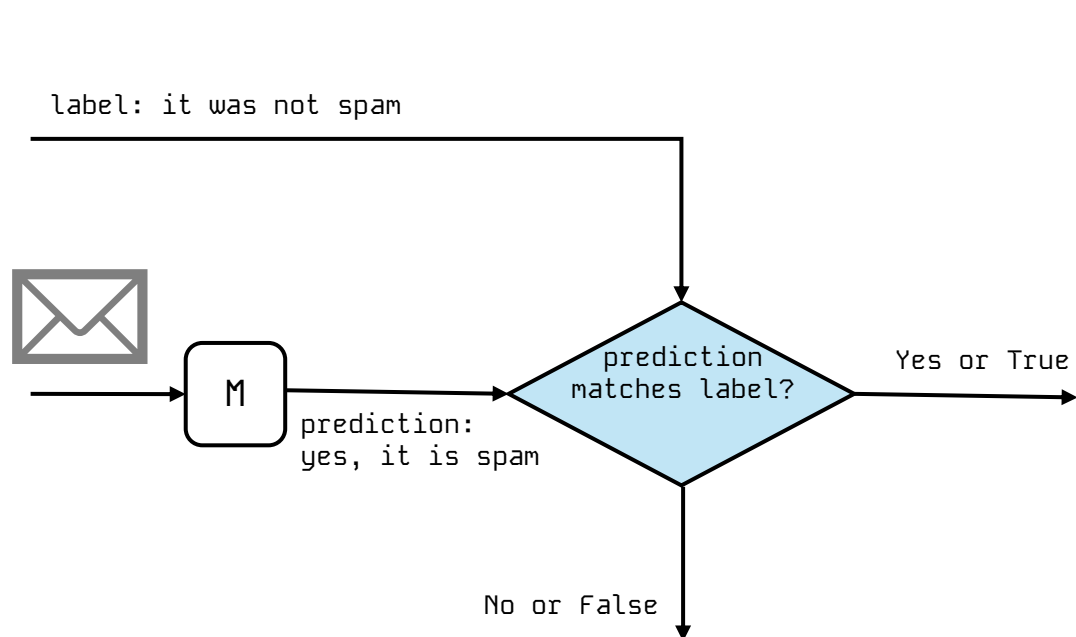


	It was spam, for real	It was not spam
Yes, it is spam		
No, it is not spam		

From Harvard CS109B

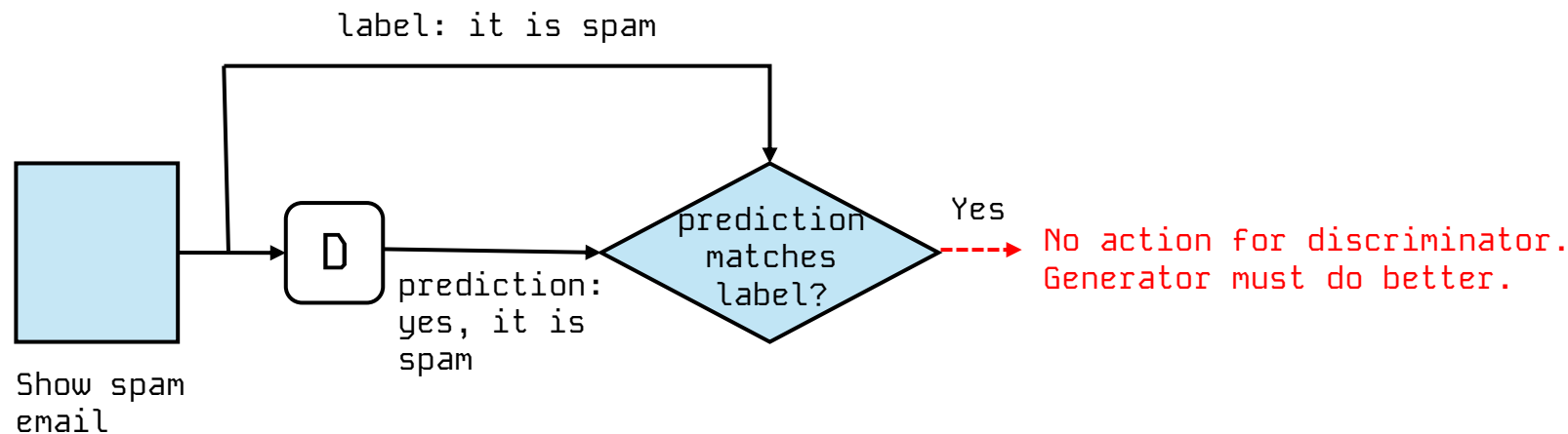
Generative Adversarial Networks

Understanding confusion matrix through an example



SPAM: POSITIVE
TRUE/FALSE: If prediction and true match do not match
POSITIVE/NEGATIVE: Prediction class

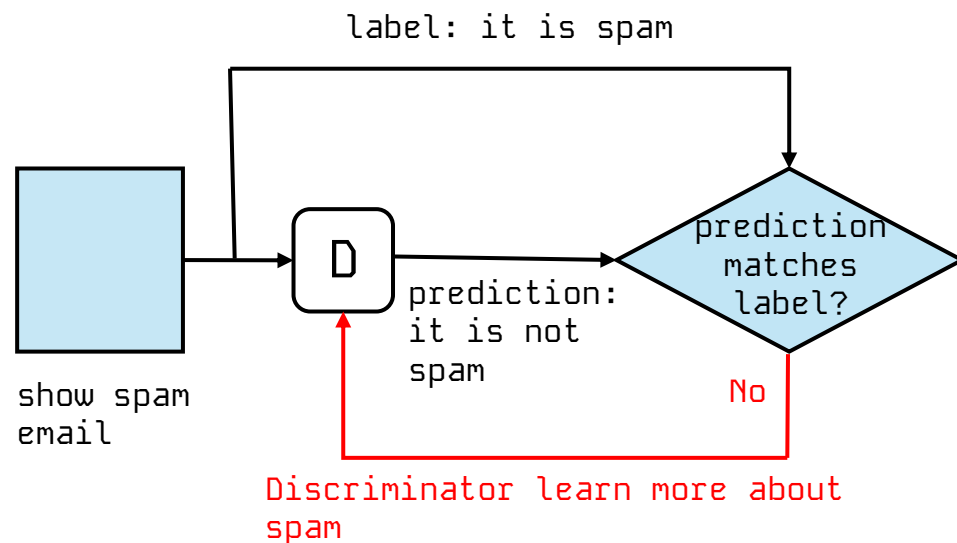
From Harvard CS109B



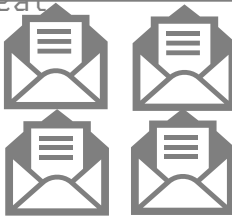
True positive (TP): the discriminator see a spam and predicts correctly. No need for further actions for discriminator. Generator must do a better job.

	It was spam, for real	It was not spam
Yes, it is spam		
No, it is not spam		

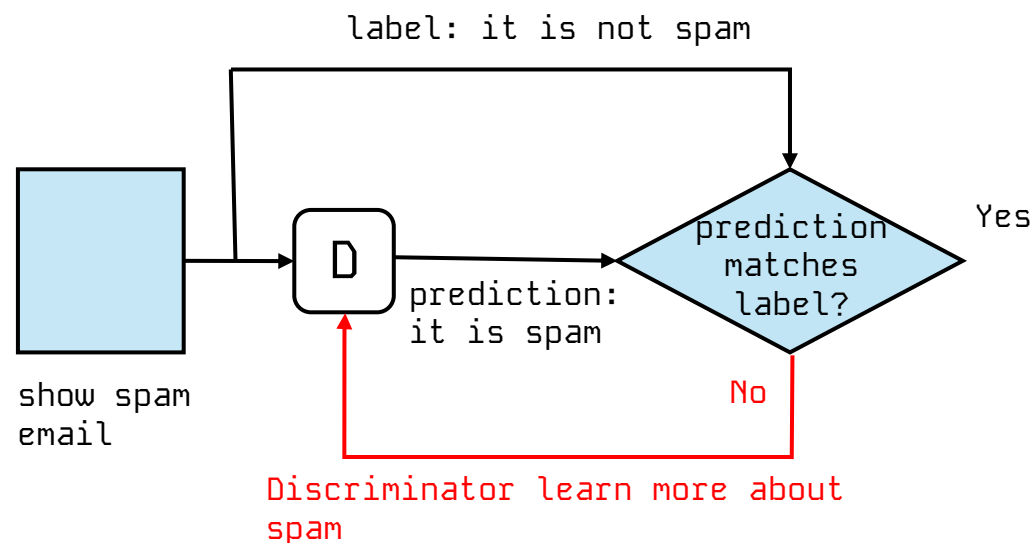
From Harvard CS109B



False Negative (FN): the discriminator see an email and predict it as spam even though it is not. The discriminator learn more

	It was spam, for real	It was not spam
Yes, it is spam		
No, it is not spam		

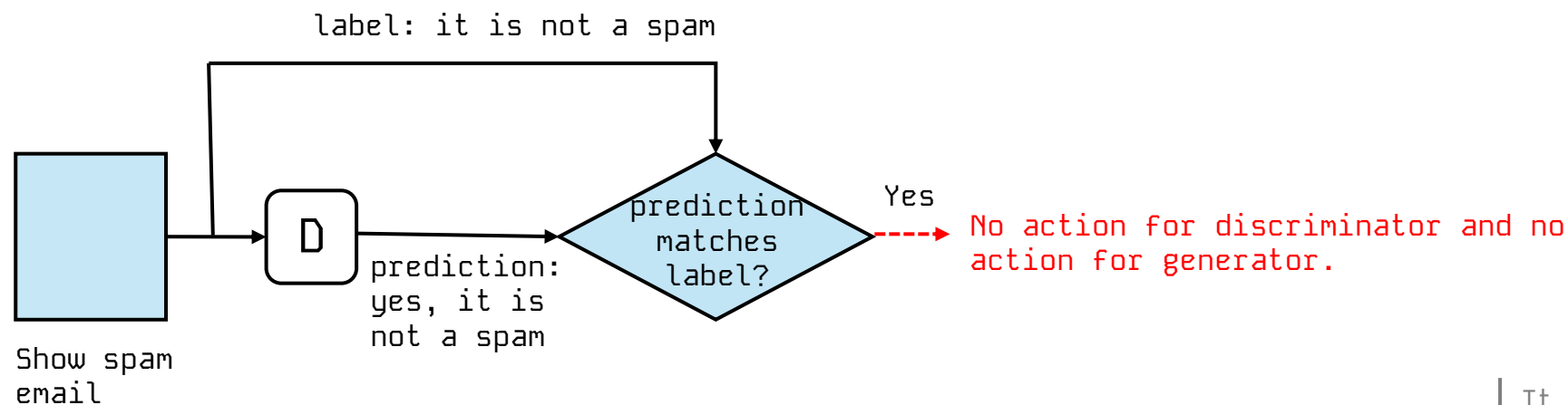
From Harvard CS109B



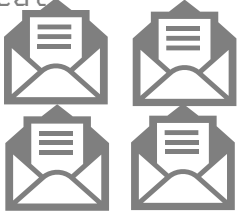
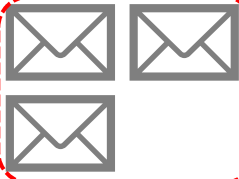
False positive (FP): generator try to fool discriminator and the discriminator fails. The generator succeeded and the discriminator is forced to improve.

	It was spam, for real	It was not spam
Yes, it is spam		
No, it is not spam		

From Harvard CS109B



True negative (TN): generator try to fool discriminator, however the discriminator predict correctly. The generator learn what was wrong and try something else.

	It was spam, for real	It was not spam
Yes, it is spam		
No, it is not spam		

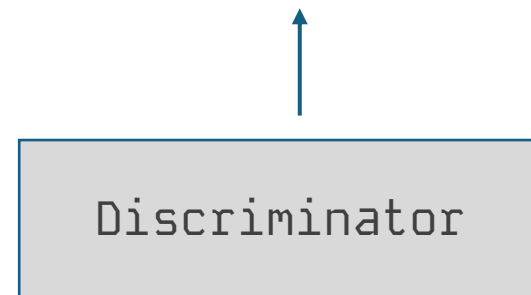
From Harvard CS109B

The Discriminator

The discriminator is very simple. It takes a sample as input, and its output is a single value that reports the network's confidence that the input is from the training set, rather than being a fake.

There are not many restrictions on what the discriminator is.

confidence that sample is real



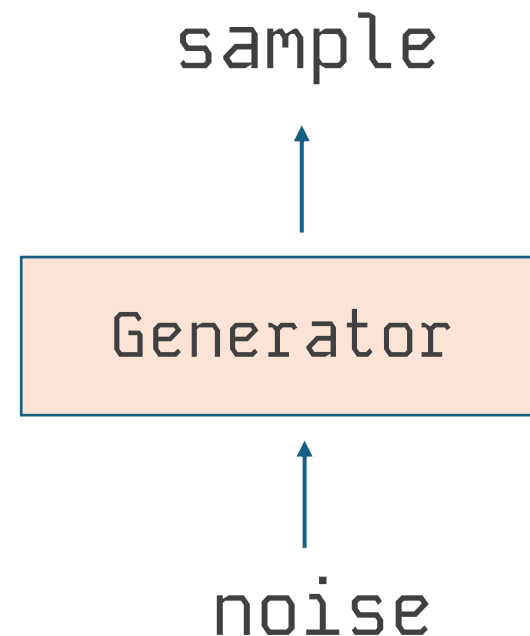
sample

From Harvard CS109B

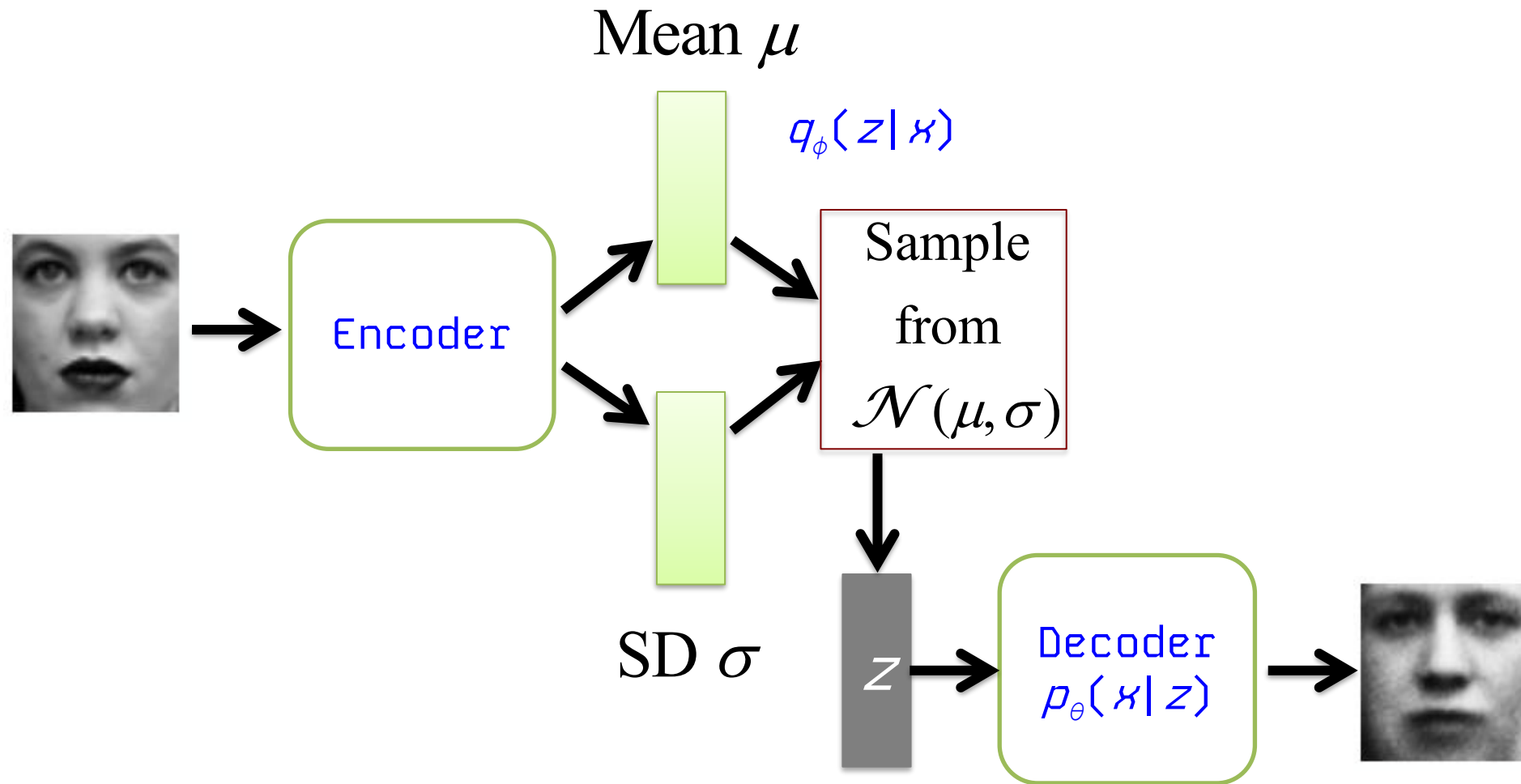
The Generator

The generator takes as input a bunch of random numbers.

If we build our generator to be deterministic, then the same input will always produce the same output. In that sense we can think of the input values as latent variables. But here the latent variables weren't discovered by analyzing the input, as they were for the VAE.

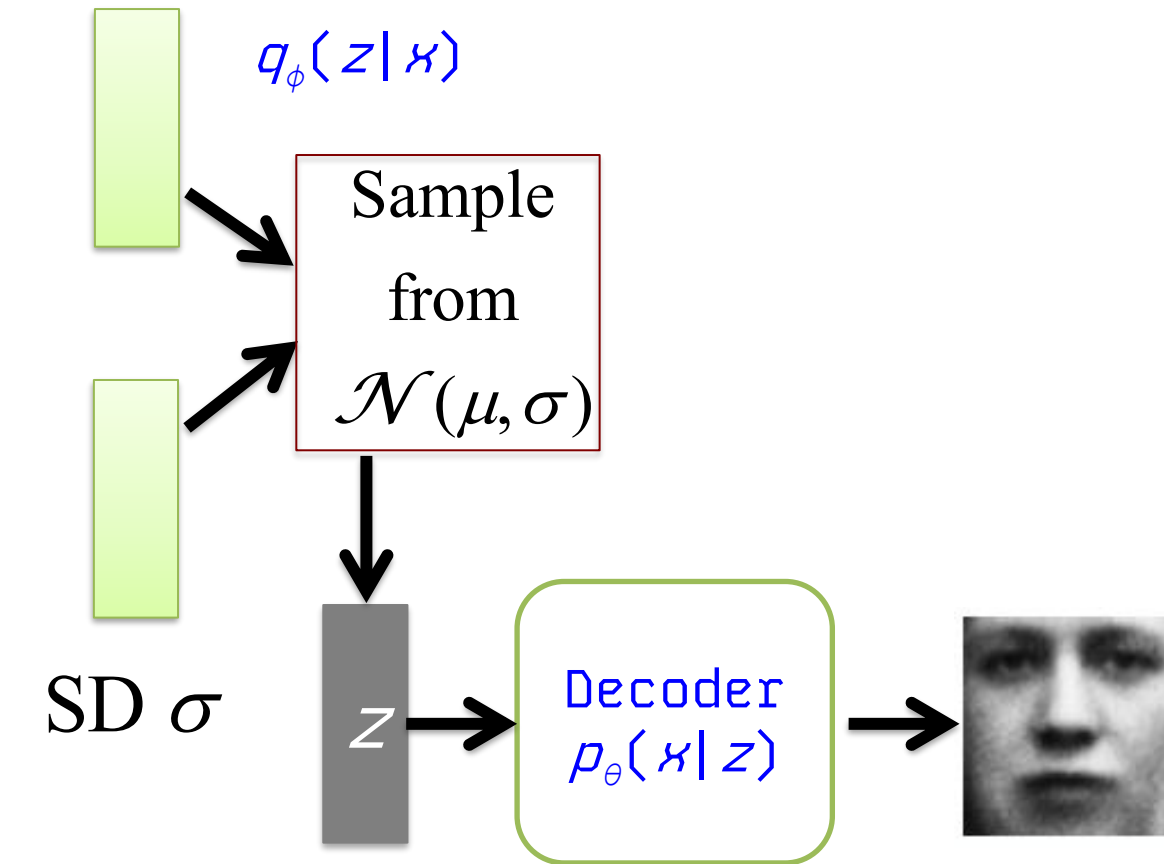


From Harvard CS109B

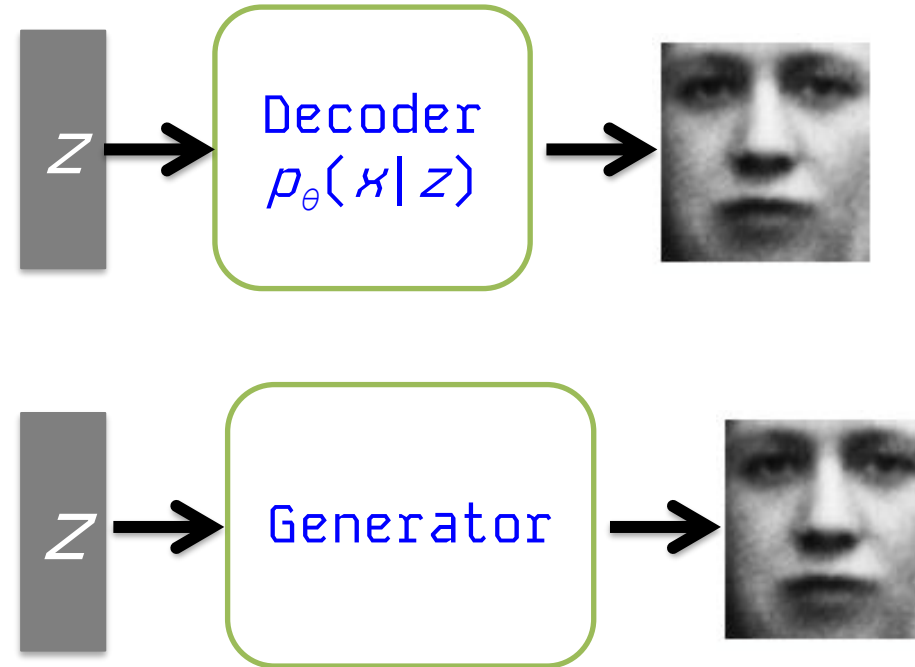


From Harvard CS109B

Mean μ



From Harvard CS109B

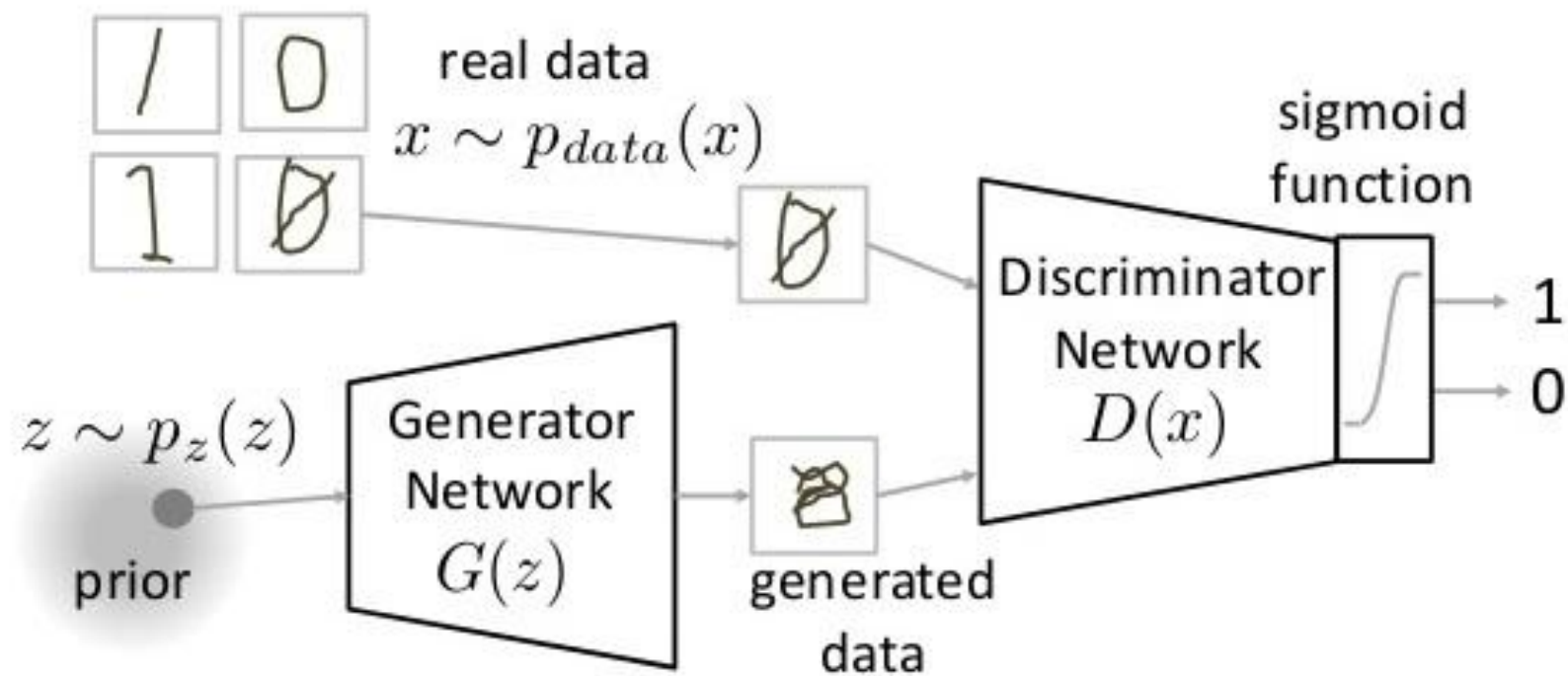


From Harvard CS109B

Learning

The process – known as Learning Round - accomplishes three jobs:

1. The discriminator learns to identify features that characterize a real sample
2. The discriminator learns to identify features that reveal a fake sample
3. The generator learns how to avoid including the features that the discriminator has learned to spot



Min-Max Cost

Discriminator seeks to *Discriminator* wants
predict 1 on training to predict 0 on samples
samples generated by G

$$V(\theta^D, \theta^G) = \mathbf{E}_{x \sim p_{data}} \log D(x) + \mathbf{E}_z \log(1 - D(G(z)))$$

Generator wants D to not distinguish between
original and generated samples!

$$\min_{\theta^G} \max_{\theta^D} J(\theta^G, \theta^D)$$

Generative Adversarial Networks

discriminative

true dist.

generative

x

z

(a)

latent space

(b)

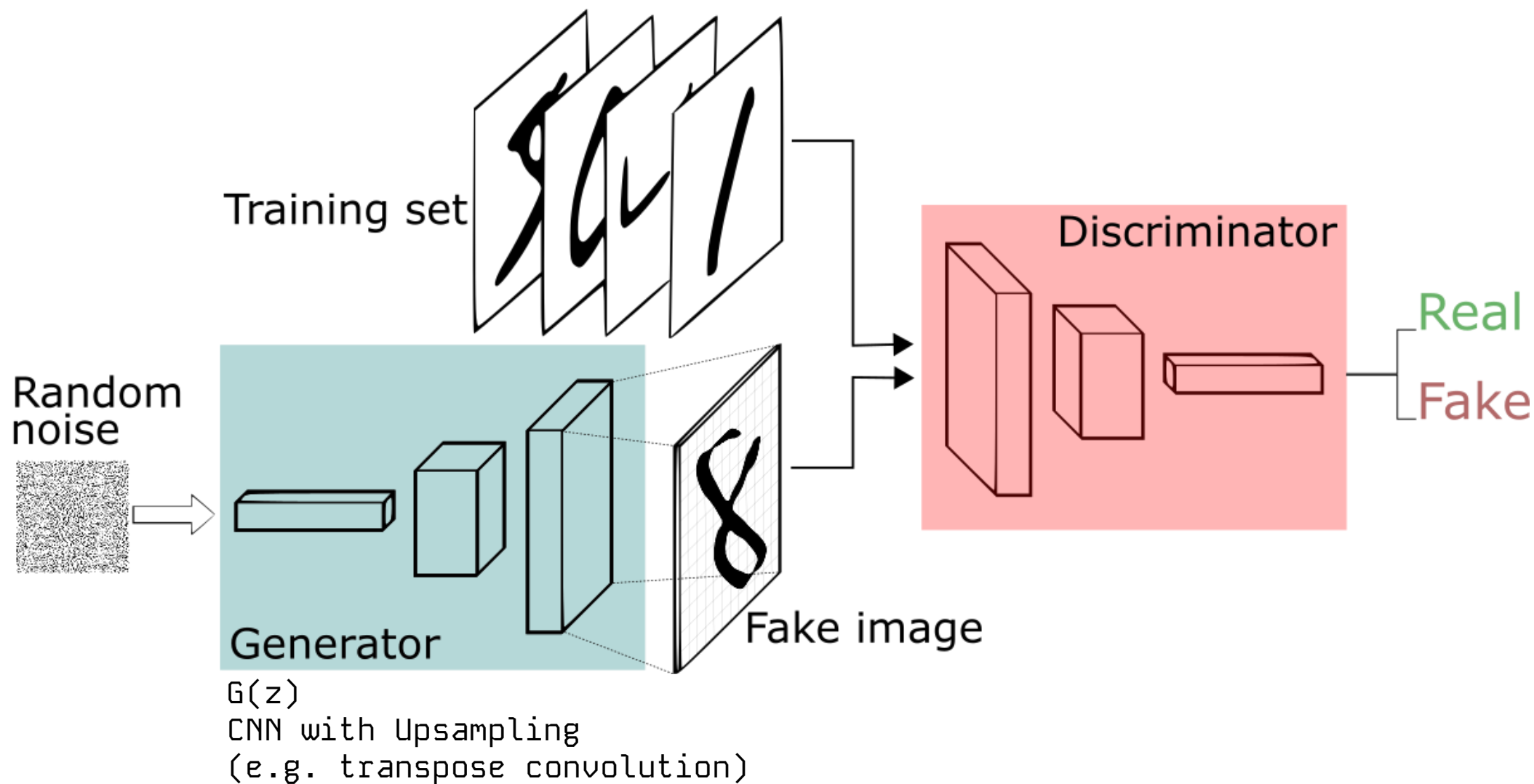
(c)

(d)

Goodfellow et al., 2014

SUPERVISED 8

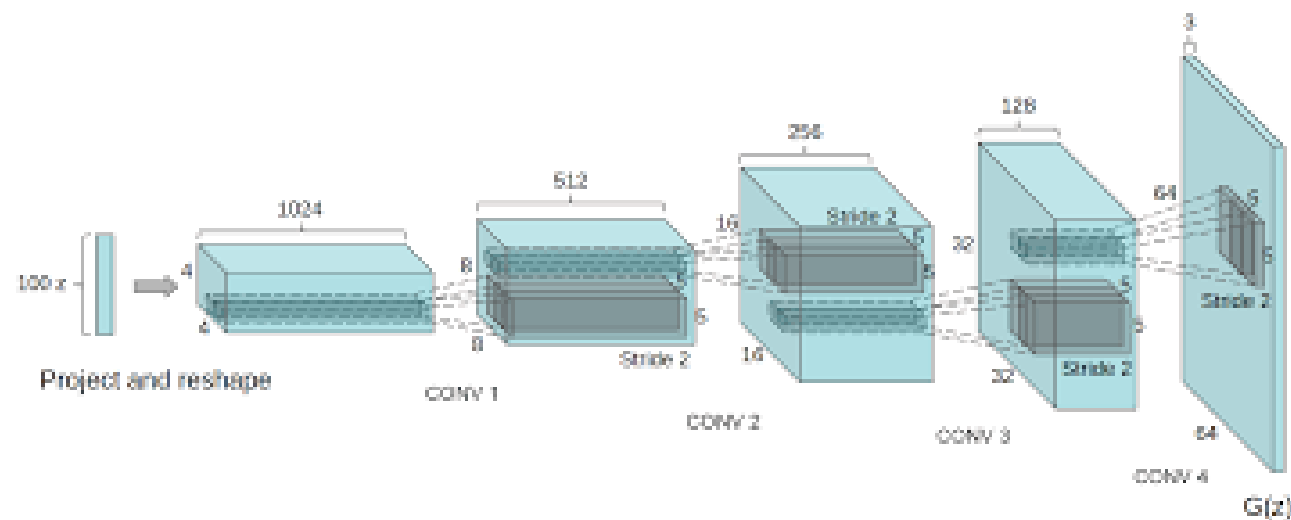
SUPERVISED 9



Deep Convolutional GAN (DCGAN)

- Alex Radford et al. 2016

- Eliminate fully connected layers.
- Max Pooling BAD! Replace all max pooling with convolutional stride
- Use transposed convolution for upsampling.
- Use Batch normalization



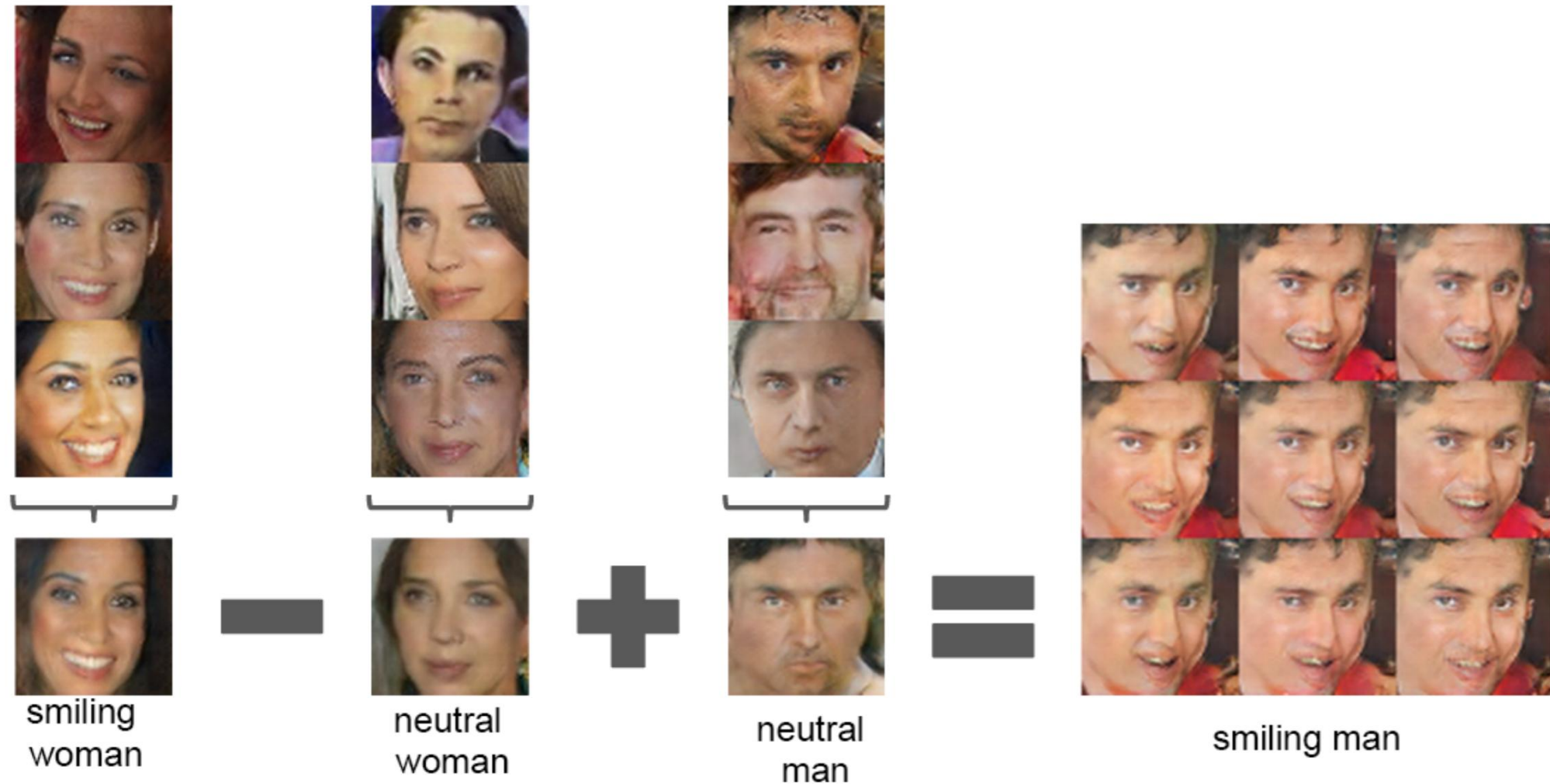
Generative Adversarial Networks



DCGAN, Radford et al., 2016

SUPERVISED 8

SUPERVISED 9



DCGAN, Radford et al., 2016

VAE



GAN



Larsen et al., 2016

GANs are hard to train, and require tricks to work well

e.g.

- Use Leaky ReLU instead of ReLU
- Use Dropout layer in discriminator layer
- Draw random latent vectors from normal distribution
- Use transpose convolution with kernel size divisible by stride (to avoid checkerboard patterns)
- Use tanh activation functions as the last layer for the generator (such that pixel colors $\in [-1,1]$)
- Smoothen real/fake labels (potentially with noise)
- Avoid max pooling layers

NIPS 2016 Tutorial: Generative Adversarial Networks

Ian Goodfellow

OpenAI, ian@openai.com

Abstract

This report summarizes the tutorial presented by the author at NIPS 2016 on *generative adversarial networks* (GANs). The tutorial describes: (1) Why generative modeling is a topic worth studying, (2) how generative models work, and how GANs compare to other generative models, (3) the details of how GANs work, (4) research frontiers in GANs, and (5) state-of-the-art image models that combine GANs with other methods. Finally, the tutorial contains three exercises for readers to complete, and the solutions to these exercises.

Introduction

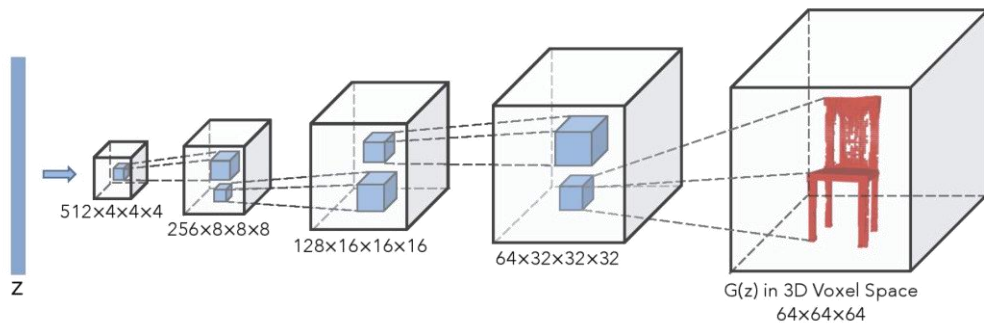
This report¹ summarizes the content of the NIPS 2016 tutorial on *generative adversarial networks* (GANs) (Goodfellow *et al.*, 2014b). The tutorial was designed primarily to ensure that it answered most of the questions asked by audience members ahead of time, in order to make sure that the tutorial would be as useful as possible to the audience. This tutorial is not intended to be a comprehensive review of the field of GANs; many excellent papers are not described here, simply because they were not relevant to answering the most frequent questions, and because the tutorial was delivered as a two hour oral presentation and did not have unlimited time cover all subjects.

The tutorial describes: (1) Why generative modeling is a topic worth studying, (2) how generative models work, and how GANs compare to other generative models, (3) the details of how GANs work, (4) research frontiers in GANs, and (5) state-of-the-art image models that combine GANs with other methods. Finally, the tutorial contains three exercises for readers to complete, and the solutions to these exercises.

The slides for the tutorial are available in PDF and Keynote format at the following URLs:

<http://www.iangoodfellow.com/slides/2016-12-04-NIPS.pdf>

¹This is the arxiv.org version of this tutorial. Some graphics have been compressed to respect arxiv.org's 10MB limit on paper size, and do not reflect the full image quality.



3D GANs

approach:

Use voxel-based shape representations

>>> a shape is an image with many channels

>>> use similar strategy as DCGAN

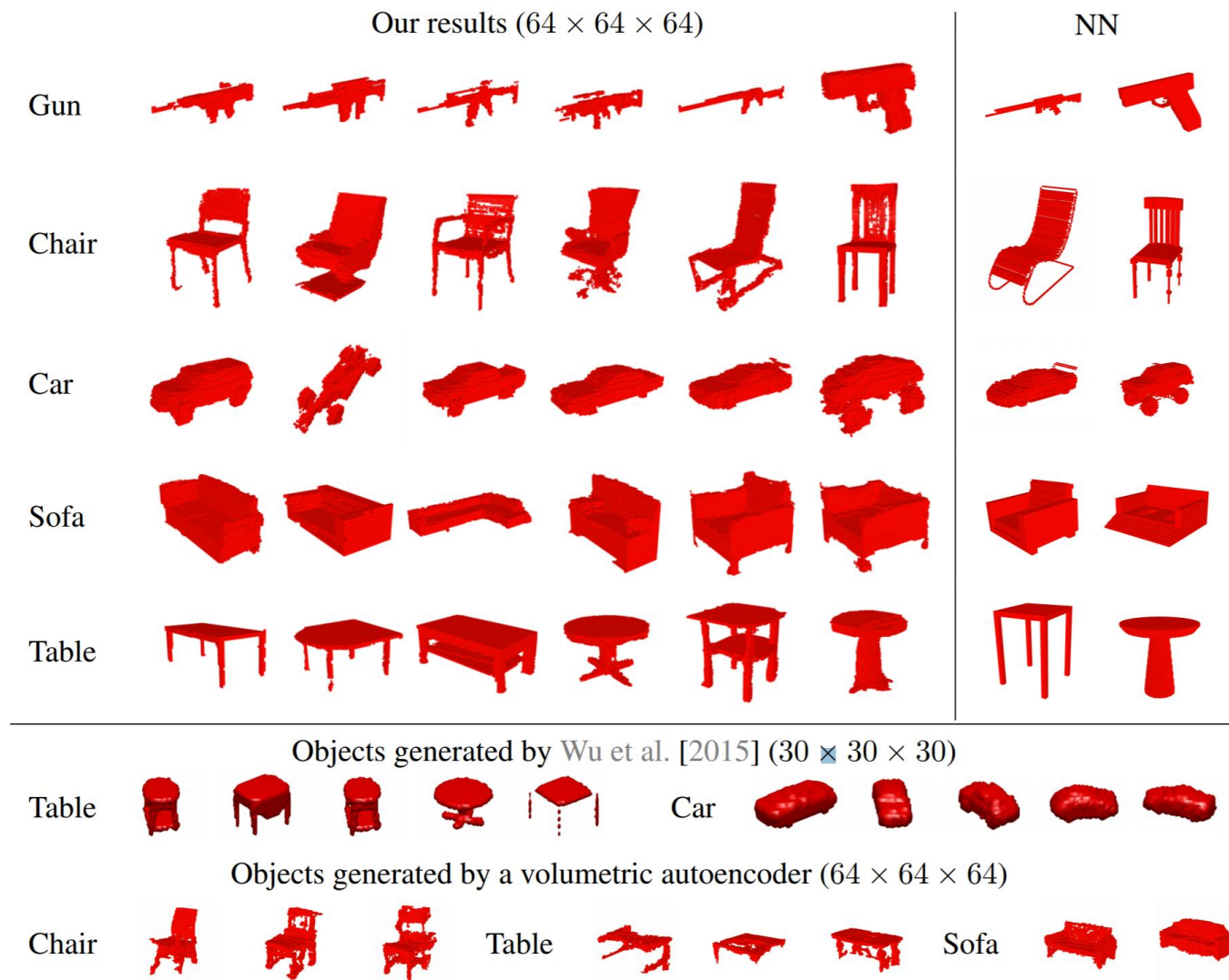
output of generator network:

$64 \times 64 \times 64$ voxels with 0-1 density/opacity
(on/off)

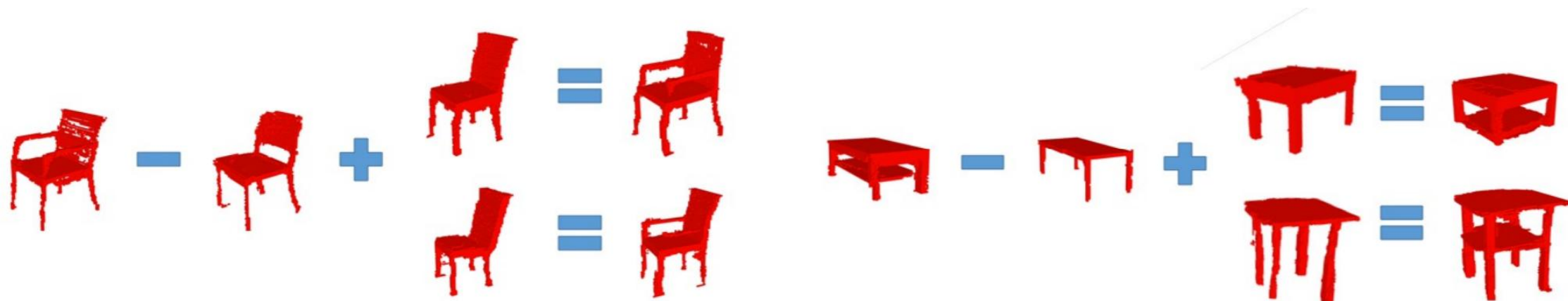
trained on voxelized representations of
shapes from ShapeNet dataset

Code, models, datasets at
<http://3dgan.csail.mit.edu/>

Wu et al., 2016



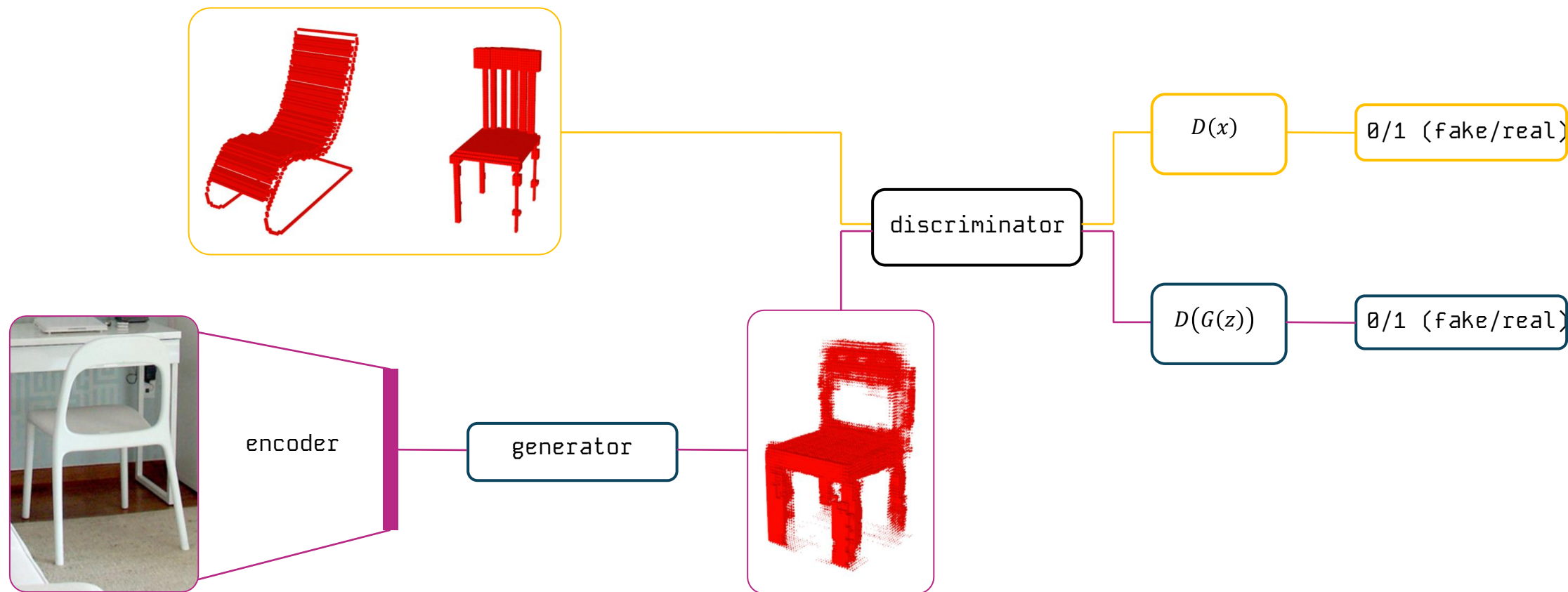
Wu et al., 2016



Wu et al., 2016

SUPERVISED 8

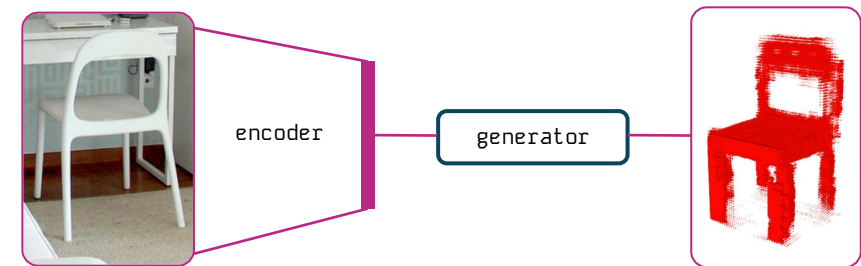
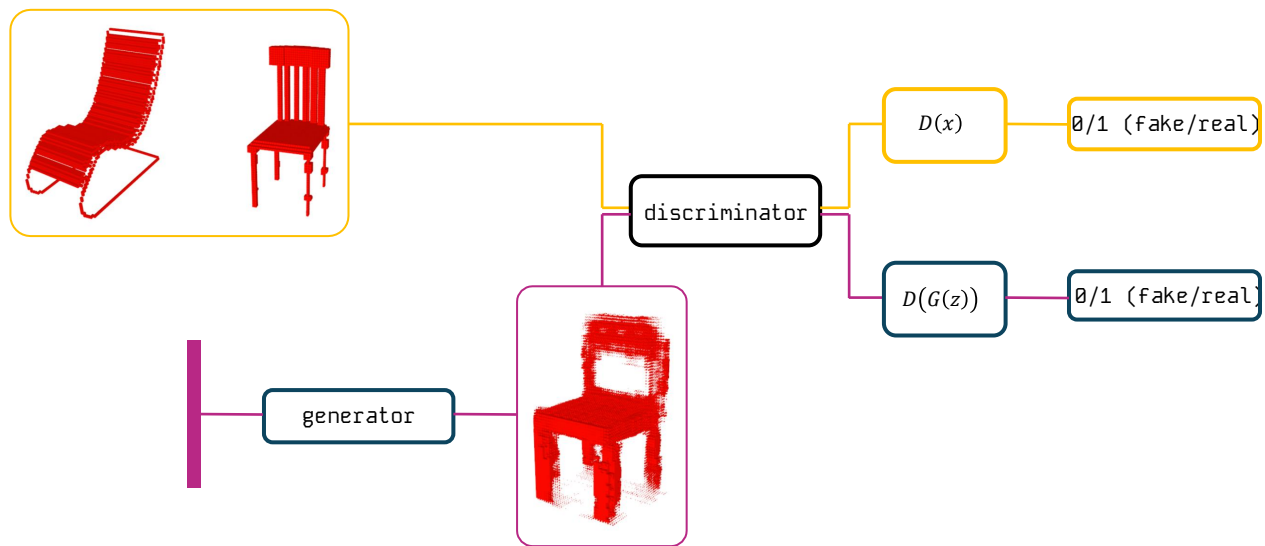
SUPERVISED 9



training: approach 1

train 3 networks simultaneously > loss becomes $L = L_{GAN} + L_{VAE} = L_{GAN} + \alpha_1 L_{KL} + \alpha_2 L_{reconstruction}$

Wu et al., 2016



Wu et al., 2016

Labels to Street Scene

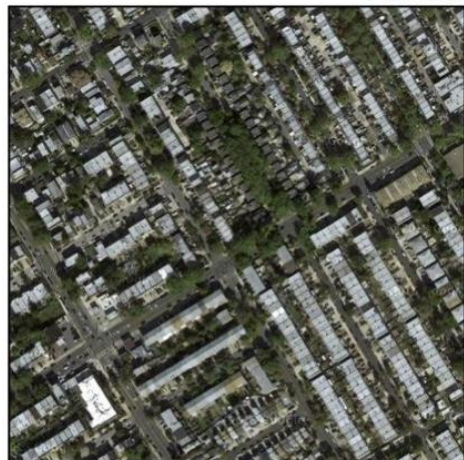


input



output

Aerial to Map

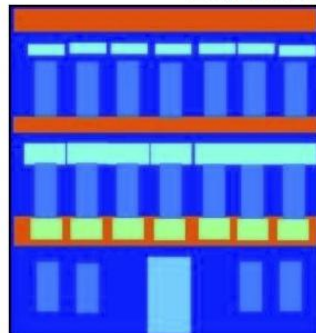


input



output

Labels to Facade



input



output

BW to Color



input



output

Day to Night



input



output

Edges to Photo



input



output

Philip Isola

<https://poloclub.github.io/ganlab/>

Google Colab
<https://bit.ly/BPS5231-L9>

Recurrent Neural Networks (LSTM, GRU)

Reinforcement Learning

Transformers

Diffusion

and many more

Due 19 November 2pm

Deliverables

1. Code or Repo
2. 8-pages Write-up
 - Conference Paper Format (<https://www.asim2024.org/call-for-papers/>)
3. Slides (5 mins presentation, 3 minutes Q&A)
4. 6-10 Graphics for publication (book)
 - white background best (can use in slides too)

Final Presentation

19 Nov

Thank You !



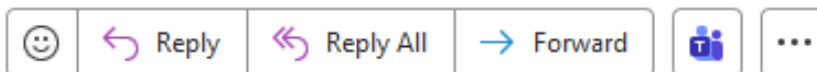
SUPERVISED 8

SUPERVISED 9

Exercise for Student Feedback 2510



NUS Student Feedback <studentfeedback@se.i
To Yu Qian Ang



Fri 10/24/2025 12:12 AM

Dear Yu Qian Ang,

Every semester, NUS gathers feedback from students on how they experienced teaching in the courses they took and are useful for purposes of continual improvement.

The Student Feedback Exercise for your classes has just started.

[Click here to check the list of classes and to monitor the response rate](#)

The link above provides an overview of the classes you taught this semester, for which feedback is being gathered. Please **ensure that your classes are reflected accurately**. You can also monitor the response rates of your classes.

You could also consider taking the following steps to encourage students to provide feedback to improve your response rate:

- Provide time in class for students to complete the evaluations (preferably at the start of the class)
- Share with your students how you would use the student feedback
- You may consider using the [“In-class instructions to students” slide](#) or share the [infographic](#) to emphasise the importance of providing effective feedback.
- Monitor your response rate from the automated email sent to you or via Canvas during the SF exercise

L09.1

Supervised 8

Autoencoders

Variational Autoencoders

L09.2

Supervised 9

Generative Adversarial Networks